

# Dynamic Group Signatures with Verifier-Local Revocation

Callum London, Daniel Gardham, Constantin Cătălin Drăgan  
Surrey Centre for Cyber Security, University of Surrey, Guildford, United Kingdom  
{c.london, daniel.gardham, c.dragan}@surrey.ac.uk

**Abstract**—Group Signatures are fundamental cryptographic primitives that allow users to sign a message on behalf of a predefined set of users, curated by the group manager. The security properties ensure that members of the group can sign anonymously and without fear of being framed. In dynamic group signatures, the group manager has finer-grained control over group updates while ensuring membership privacy (i.e., hiding when users join and leave). The only known scheme that achieves standard security properties and membership privacy has been proposed by Backes et al. CCS 2019. However, they rely on an inefficient revocation mechanism that re-issues credentials to all active members during every group update, and users have to rely on a secure and private channel to join the group.

In this paper, we introduce a dynamic group signature that supports verifier local revocation, while achieving strong security properties, including membership privacy for users joining over a public channel. Moreover, when our scheme is paired with structure-preserving signatures over equivalence class it enjoys a smaller signature size compared to Backes et al. Finally, as a stand-alone contribution we extend the primitive Asynchronous Remote Key Generation (Frymann et al. CCS 2020) with trapdoors and introduce new security properties to capture this new functionality, which is fundamental to the design of our revocation mechanism.

**Index Terms**—dynamic group signatures, membership privacy, verifier-local revocation

## 1. Introduction

Group signatures, introduced by Chaum and Van Heyst [1], allow a group manager to delegate signing capabilities to a set of group members (i.e., signers). They are often used in commercial settings where employees sign on behalf of the company without needing to bear public responsibility, e.g., anonymous electronic cash payment [2], [3]. This is ensured by *unforgeability* [4], [5] where only members of that group can produce valid signatures, *anonymity* [5], [6], [7] such that the signature does not reveal the identity of the signer, and by *traceability* [5], [6], [7] where the group manager can prove that a signature was produced by a specific user. A specific example of group signatures in an organisational setting is DocuSign’s *Signing Groups* feature [8], but has seen uses in other domains such as vehicle-to-vehicle communication [9], [10].

Unforgeability is often enhanced to a stronger notion called *non-frameability* [5], [6] where honest signers cannot be framed to have produced a signature even if all other signers and group manager being corrupted. When attackers have access to leaked signer keys, then anonymity is modeled as *full anonymity* [4], [11], while limiting the adversary by hiding traced signatures gives the weakened definition of *selfless anonymity* [12], [13]. Any scheme that satisfies full-anonymity also satisfies anonymity. *Full traceability* [4], [11] models colluding signers and in [4] it has been shown to imply traceability and non-frameability.

Depending on how groups membership is established, we have *static* [1], [4], [12], [14], [11]—fixed and set at setup, and *dynamic* [6], [7]—users can join and leave the group. *Partial dynamic* [5], [13] group signatures are a special case of dynamic, where the role of group manager is divided into 2 entities - one for managing membership (e.g., join and leave) and another for tracing signatures. To strengthen privacy of dynamic group signatures Backes et al. [7] have formalised *membership privacy* to model an adversary’s ability to identify when users join or leave the group.

In the context of (partial) dynamic group signatures the joining phase can be done over *public channel* [15], [16], [13] or over secure and *private channel* [5], [7].

To enable appropriate removal of users from groups, dynamic group signatures rely on well-established revocation techniques, e.g., revocation lists (i.e., RL - *public revocation list* [6] and VLR - *verified-local revocation* [12], [14], [13]), and *secret credentials* [7]. Schemes based on secret credentials incorporate a huge overhead as it relies on re-issuing credentials to all signers (except those revoked). Schemes based on RL rely on the signer having up-to-date revocation lists and showing that their signing key has not been revoked (e.g., using zero-knowledge proofs), while for VLR the verifier must have up-to-date information and check some anonymised signer token is not in the revocation list. Using *epochs* that limit frequency of re-issuing credentials or size of revocation list can help mitigate some of the overhead complexity.

Backes et al. [7] have shown how to design generic group signatures that satisfy membership privacy, anonymity, traceability and non-frameability. However, their design requires a *private channel* to perform the join and relies on *inefficient secret credential revocation* methods.

*Open problem:* can generic dynamic group signatures be designed while ensuring: (1) a more efficient revocation

*mechanism, (2) a join phase over public channels, and (3) maintaining the same security guarantees: anonymity, traceability, non-frameability and membership privacy?*

We note that such a primitive would find uses in, for example, Direct Anonymous Attestation (which is a type of group signature) used within Trusted Platform Modules (TPMs). Kumar et al. [17] propose a DAA-based subscription service that uses the local revocation mechanism of the verifier to shift computational overhead away from the low-compute TPM and to the verification server. Membership privacy would enhance the privacy guarantees in such context by preventing the identification of users as they join or leave the subscription service. Another example is for the use in privacy-enhancing crowd sensing [18] which involves individuals using mobile devices (such as smartphones or wearables) to collectively gather and share data to analyze or predict processes of common interest. Again, a verifier-local revocation mechanism is used to reduce computational burden on smart devices and onto the verifying data-collection server. Strong dynamic privacy properties prevent a user from being associated with its contributed data, and on the other hand, the tracing ability of the group manager can be used to identify those maliciously contributing false data, which is known problem in crowd sensing at scale [19].

## 1.1. Contributions

In this paper, we answer the open problem in the affirmative and present DGS-VLR—a generic dynamic group signature that supports verifier local revocation, and offers an efficient group membership update procedure. We achieve this using a novel two-step approach that uses unlinkable ‘intermediate’ keys that can be revoked without compromising the membership privacy of the group member. We give a generic construction based on Asynchronous Remote Key Generation [20], Signatures with Flexible Public Keys [11], Digital signatures [21], and a Non-interactive Zero-Knowledge Proof (NIZK) which we instantiate with a Structure-Preserving Signature over Equivalence Classes (SPS-EQ) [22].

This is comparable with the state-of-the-art by Backes et al. [7] as we achieve the same strong security properties (anonymity, traceability, non-frameability and membership privacy) while reducing the complexity of the group update procedure to be linear in the number of additions/revocations, instead of linear in total group members. Moreover, we provide group-based instantiations of the building blocks that allows our construction to achieve smaller signature sizes (cf. Table 1 and Table 2). In particular, our signature has size:  $11G_1 + 5G_2 + 512\text{bits}$ . An additional property of our scheme is that it allows users to join groups over public channels without interactive communication with the group manager.

Furthermore, to enable our construction, we extend Asynchronous Remote Key Generation with trapdoors that allow linking of keys under restricted conditions. This novel functionality is fundamental to the design of our efficient

revocation mechanism and could be of independent interest in other privacy-preserving primitives.

## 1.2. Related Work

We compare our scheme with other known and state-of-the-art group signatures in Table 1. We focus on the **Type** of scheme (i.e., Dynamic, partial Dynamic or Static), if they satisfy **Membership Privacy** (i.e., Yes, No), over what type of channel the **Join** procedure is performed (i.e., private, public, or non-available), the method and cost of **Revocation** (e.g., RL - public revocation list, VLR - verified-local revocation, and secret credential), the standard **Security** properties (e.g., anonymity, traceability, non-frameability), and the **Size of the Signature**.

The cost of revocation for  $r$  user revocations (i.e., calling  $r$  times the Revoke algorithm) for secret credentials is large as it need to re-issue credentials to all  $n$  active members during each revocation -  $\mathcal{O}(r \cdot n)$ . Schemes based on revocation lists (i.e., RL and VLR) need to only compute information for the revoked signer -  $\mathcal{O}(r)$ . However, VLR may not be preferred over RL as the verifier needs to maintain an up-to-date revocation list, typically of size  $\mathcal{O}(r)$ .

Bellare et. al. [4] introduced a static group signature that satisfy stronger notions of anonymity and traceability (i.e., full-anonymity and full-traceability) that imply existing definitions of anonymity, traceability, unforgeability and non-frameability. This has been extended in [5] to partial dynamic group signatures where two different entities have roles for joining and tracing, while only satisfying anonymity, traceability and non-frameability. Both [4] and [5] do not consider any mechanism for revocation. Additionally, they do not provide any instantiation for their primitives (i.e., what type of one-time signature they use) as such it’s not possible to estimate the size of the signature.

Boneh and Shacham [12] introduced a static group signature with a verification mechanism called verifier-local revocation (VLR) where the verifier assumes responsibility for checking revocation. However, their scheme only satisfies a weaker notion of anonymity, called selfless anonymity, where the adversary doesn’t have access to the signer’s key. This has been extended to full-anonymity in Ishida et al. [14] via the Sign-then-Encrypt paradigm. Both [12] and [14] have an implicit traceability due to the group manager generating signing keys and revocation tokens for users during joining phase. The signature size for [12] is small, while for [14] it can’t be estimated (e.g., they rely on a complex NIZK and do not show how to instantiate it).

Bootle et. al. [6] introduces a unified model for dynamic group signatures and their security properties (i.e., full anonymity, anonymity, selfless anonymity, full traceability, traceability, and non-frameability). Our scheme DGS-VLR relies on the same definitions for anonymity, traceability and non-frameability. Additionally, Bootle et. al. show that the schemes proposed by Libert et al. [15], Libert et al. [23] and Nakanishi et al. [16] satisfy their new security models and are anonymous, traceable and non-frameable. The only significant difference between [15] and [23] is a drastic

Table 1. COMPARISON OF GROUP SIGNATURE WHERE “-” MEANS NON-AVAILABLE, VLR = VERIFIER LOCAL REVOCATION, RL = REVOCATION LIST, FA = FULL ANONYMITY, SA = SELFLESS ANONYMITY, A = ANONYMITY, T = TRACEABLE, NF = NON-FRAMEABLE, U = UNFORGEABLE, FT = FULL TRACEABILITY. WE USE  $r$  AS THE NUMBER OF REVOKED USERS AND  $n$  THE SIZE OF THE GROUP.

| Scheme                     | Type            | Membership Privacy | Join          | Revocation        |                          | Security        | Signature Size                          |
|----------------------------|-----------------|--------------------|---------------|-------------------|--------------------------|-----------------|-----------------------------------------|
|                            |                 |                    |               | Type              | Cost                     |                 |                                         |
| Bellare et. al. [4]        | Static          | No                 | -             | -                 | -                        | fA, fT, NF      | -                                       |
| Bellare et. al. [5]        | partial Dynamic | No                 | private       | -                 | -                        | A, T, NF        | -                                       |
| Boneh et al. [12]          | Static          | No                 | -             | VLR               | $\mathcal{O}(r)$         | sA, T, U        | $2G_1 + 5Z_q$                           |
| Ishida et al. [14]         | Static          | No                 | -             | VLR               | $\mathcal{O}(r)$         | fA, T, U        | -                                       |
| Libert et al [15] (SD)     | partial Dynamic | No                 | public        | RL                | $\mathcal{O}(r)$         | A, T, NF        | $180G_1$                                |
| Nakanishi et al [16]       | Dynamic         | No                 | public        | RL                | $\mathcal{O}(r)$         | A, T, NF        | $10G_1 + 3G_2 + 30Z_q$                  |
| Bichsel et. al. [13]       | Dynamic         | No                 | public        | VLR               | $\mathcal{O}(r)$         | sA, T, NF       | $4G_1 + 5Z_q$                           |
| Backes et. al. [11]        | Static          | No                 | -             | -                 | -                        | fA, fT, NF      | $20G_1 + 5G_2$                          |
| Backes et. al. [7]         | Dynamic         | Yes                | private       | secret credential | $\mathcal{O}(r \cdot n)$ | A, T, NF        | $28G_1 + 15G_2 + 512\text{bits}$        |
| <b>This Work w/ NIZK</b>   | <b>Dynamic</b>  | <b>Yes</b>         | <b>public</b> | <b>VLR</b>        | $\mathcal{O}(r)$         | <b>A, T, NF</b> | $(4n + 21)G_1 + 19G_2 + 512\text{bits}$ |
| <b>This Work w/ SPS-EQ</b> | <b>Dynamic</b>  | <b>Yes</b>         | <b>public</b> | <b>VLR</b>        | $\mathcal{O}(r)$         | <b>A, T, NF</b> | $11G_1 + 5G_2 + 512\text{bits}$         |

reduction in the size of the public key. All three schemes [15], [23], [16] use revocation lists with [16] producing a smaller signature size compared to [15], [23].

An alternative approach to partial-dynamic group signatures is proposed by Bichsel et. al. [13] relies on re-randomisable signatures. They show their scheme is non-frameable, traceable, and selfless-anonymous. This has been extended by Backes et. al. [11] that used Signatures with Flexible Public Keys (FPK) to obtain static group signatures with smaller signature size while ensuring full-anonymity and full-traceability. Another extension by Backes et. al. [7] focused on introducing membership privacy for when users join/leave the group (cf. Table 1) and ensuring anonymity, traceability and non-frameability. However, revocation requires re-issuing credentials to all group users, for each member that is revoked, i.e. has complexity  $\mathcal{O}(r \cdot n)$ .

In this work, we achieve strong security properties: anonymity, traceability, non-frameability and membership privacy, whilst also supporting new more efficient revocation methods whose complexity is only linear in the number of revoked users, i.e.  $\mathcal{O}(r)$ .

## 2. Preliminaries

In this section we define the relevant cryptographic building block that we used in our construction our DGS-VLR scheme: Asynchronous Remote Key Generation [20], Signatures with Flexible Public Keys [11], and Non-interactive Zero-Knowledge [24].

### 2.1. Asynchronous Remote Key Generation

An Asynchronous Remote Key Generation (ARKG) protocol [20] allows a party to create a public key on behalf of another, without exchanging any private information. The issuing party uses an existing long-term public key  $pk$  belonging to the receiver to derive a new public key  $pk'$ . In a one-round communication over a public channel the issuer sends some credentials  $cred$ , from which the receiver is able to derive a corresponding secret key  $sk'$ , using knowledge of their secret key  $sk$ . There are two security properties, *pk-unlinkability* that ensures that  $pk$

and  $pk'$  cannot be linked, and *sk-security* which ensures that only a party with knowledge of  $sk$  can derive a valid  $sk'$ . There are 4 flavours of this property that captures an adversary’s abilities: in this paper we make use of variant *malicious strong* key secrecy. ARKG has already seen uses in privacy-preserving delegatable cryptography by enabling proxy signatures with unlinkable warrants for use in the WebAuthn ecosystem [25], whilst the original work focused on discrete-log key pairs, there have been extensions to pairings [26] and even for lattices [27], [28], [29].

We introduce a variant of the ARKG scheme where  $pk$  and  $pk'$  are linked under specific conditions, i.e., using a secret token  $\tau$  generated as an additional output by the DerivePK algorithm. We introduce a new algorithm ChkLink to link two public keys and define a new security property, *linking soundness*, over this new algorithm.

**Definition 1 (Asynchronous Remote Key Generation [20]).** An ARKG scheme is a tuple (Setup, KeyGen, DerivePK, DeriveSK, ChkKeys, ChkLink) defined as:

- $pp \leftarrow \text{Setup}(\lambda)$  takes as input the security parameter  $\lambda$  and outputs a set of public parameters  $pp$ . The public parameters are implicit input to all other algorithms
- $(sk, pk) \leftarrow \text{KeyGen}()$  on input  $pp$  samples a private-public key pair  $(sk, pk)$ .
- $(pk', cred, \tau) \leftarrow \text{DerivePK}(pk, aux)$  produces a new public key  $pk'$ , along with credentials  $cred$  and a link  $\tau$ , from  $pk$  with some (potentially empty) auxiliary input  $aux$ . For simplicity, we abuse notation and use  $\text{DerivePK}(pk)$  when no  $aux$  is provided.
- $sk' \leftarrow \text{DeriveSK}(sk, cred)$  computes deterministically  $sk'$  together with the information provided by  $cred$  to derive a new private key  $sk'$  corresponding to  $pk'$ . It may fail, in which case it outputs  $\perp$  instead.
- $\{0, 1\} \leftarrow \text{ChkKeys}(sk', pk')$  checks whether the given pair  $(sk', pk')$  forms a valid private-public key pair, outputting 1 if so and 0 otherwise.
- $\{0, 1\} \leftarrow \text{ChkLink}(pk, pk', cred, \tau)$  decides whether the given link  $\tau$  proves that  $pk'$  and  $cred$  were derived from  $pk$ , outputting 1 if so and 0 otherwise.

*Correctness* ensures with overwhelming probability a valid new key pair is generated and they can be linked using the

token  $\tau$ , if the protocol steps were followed. More precisely,  $1 - \text{negl}(\lambda) \leq \Pr[\text{ChkKeys}(\text{sk}', \text{pk}') = 1]$  and  $1 - \text{negl}(\lambda) \leq \Pr[\text{ChkLink}(\text{pk}, \text{pk}', \text{cred}, \tau) = 1]$  given

$$\begin{aligned} \text{pp} &\leftarrow \text{Setup}(\lambda) \\ (\text{sk}, \text{pk}) &\leftarrow \text{KeyGen}() \\ (\text{pk}', \text{cred}, \tau) &\leftarrow \text{DerivePK}(\text{pk}) \\ \text{sk}' &\leftarrow \text{DeriveSK}(\text{sk}, \text{cred}). \end{aligned}$$

The security properties *pk-unlinkability* and *sk-security* are adapted from the original work [20] (with considerations of the improved definitions of follow up works [29], [28]) to support the additional output  $\tau$  and algorithm  $\text{ChkLink}$ . We highlight that this does not change the intuition and goals, as the adversary does not have access to  $\tau$  in either experiment. Intuitively, for *pk-unlinkability* the adversary cannot distinguish between a fresh key generation process and a key derived using ARKG, while for *sk-security* the adversary can't compute the secret key matching the derived public key. The experiments are presented in the full version of this paper. We have the following advantages negligible in  $\lambda$  for an efficient adversary  $\mathcal{A}$ :

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda) &= \left| \Pr[\text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{pku}} = 1] - \frac{1}{2} \right| \text{ and} \\ \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{sk}}(\lambda) &= \Pr[\text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{sk}} = 1] \end{aligned}$$

In accordance with the adapted syntax definition, we introduce an additional property, namely the soundness of a proof of linking. This captures the probability that a derived key  $\text{pk}'$  is created from  $\text{pk}$  with link  $\tau$ , implies that the  $\text{ChkLink}$  algorithm succeeds.

**Definition 2** (Linking Soundness). ARKG satisfies *linking soundness* if for any PPT adversary  $\mathcal{A}$  the following advantage is negligible in  $\lambda$ :

$$\text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{l-sound}}(\lambda) = \Pr \left[ \begin{array}{l} \text{pp} \leftarrow \text{Setup}(\lambda) \\ (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}() \\ (\text{pk}', \text{cred}, \tau) \leftarrow \mathcal{A}(\text{pk}) \\ b \leftarrow \text{ChkLink}(\text{pk}, \text{pk}', \text{cred}, \tau) \\ \text{sk}' \leftarrow \text{DeriveSK}(\text{sk}, \text{cred}) \\ b' \leftarrow \text{ChkKeys}(\text{sk}', \text{pk}') \\ \text{return } b \wedge \neg b' \end{array} \right]$$

## 2.2. Signatures with Flexible Public Keys

Signatures with Flexible Public Keys (SFPK) [11] are reminiscent of standard digital signatures but allow a signer to change their public key while still being able to sign. This is achieved by changing keys within an equivalence class, and importantly, signatures verify against all keys in the equivalence class. An initial key pair  $(\text{sk}, \text{pk})$  may be generated with a trapdoor  $\tau$  that can be used to check whether another key  $\text{pk}'$  lies in the same class denoted  $\text{pk}' \in [\text{pk}]$ .

**Definition 3** (Signatures with Flexible Public Keys [11]). An SFPK scheme is a tuple  $(\text{Setup}, \text{KeyGen}, \text{TrapGen},$

$\text{ChgPK}, \text{ChgSK}, \text{Sign}, \text{Verify}, \text{ChkRep})$  consisting of the following algorithms.

- $\text{pp} \leftarrow \text{Setup}(\lambda)$  takes as input the security parameter  $\lambda$  and outputs a set of public parameters  $\text{pp}$ . These are an implicit input for all other algorithms.
- $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}()$  samples a private-public key pair.
- $(\text{pk}', \tau) \leftarrow \text{TrapGen}(\text{pk})$  produces a public key  $\text{pk}'$  with trapdoor  $\tau$ , associated with the equivalence class  $[\text{pk}]$ <sup>1</sup>.
- $\text{pk}' \leftarrow \text{ChgPK}(\text{pk}, r)$  returns an equivalent public key  $\text{pk}' \in [\text{pk}]$ , for a public key  $\text{pk}$  and salt  $r$ .
- $\text{sk}' \leftarrow \text{ChgSK}(\text{sk}, r)$  takes a secret key  $\text{sk}$  and salt  $r$ , returning an updated secret key  $\text{sk}'$ .
- $\sigma \leftarrow \text{Sign}(\text{sk}, m)$  uses the key  $\text{sk}$  to produce a signature  $\sigma$  on the input message  $m$ .
- $\{0, 1\} \leftarrow \text{Verify}(\text{pk}, m, \sigma)$  outputs 1 if  $(m, \sigma)$  forms a valid message-signature pair against  $\text{pk}$  and 0 otherwise.
- $\{0, 1\} \leftarrow \text{ChkRep}(\tau, \text{pk})$  outputs 1 if  $\text{pk}$  lies in the equivalence class associated with  $\tau$ , and 0 otherwise.

*Correctness* of SFPK ensures the following properties hold

- 1) Algorithms  $\text{KeyGen}$  and  $\text{TrapGen}$  produce the same distribution over key-pairs.
- 2) With overwhelming probability the following two verification holds:  $1 \leftarrow \text{Verify}(\text{pk}, m, \text{Sign}(\text{sk}, m))$  and  $1 \leftarrow \text{Verify}(\text{pk}', m, \text{Sign}(\text{sk}', m))$ , given valid key-pairs  $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\lambda)$  and  $\text{pk}' \leftarrow \text{ChgPK}(\text{pk}, r)$ ;  $\text{sk}' \leftarrow \text{ChgSK}(\text{sk}, r)$ .
- 3) With overwhelming probability the algorithm  $\text{ChkRep}(\tau, \text{pk}')$  returns 1 iff  $\text{pk}' \in [\text{pk}]$  for any  $(\text{pk}', \tau) \leftarrow \text{TrapGen}(\lambda)$ .

The security of SFPK [11] is based on *class-hiding* and *unforgeability*. For class-hiding an adversary cannot distinguish between public keys generated from different equivalence classes, allowing the key-change function to “hide” the class containing the original key. The unforgeability of SFPK captures an adversary's capability to create forgeries in the context of key-adaption. That is, the adversary needs to create a new public key  $\text{pk}^* \in [\text{pk}]$  and a forgery for that key. The formal experiments are presented in the full version of this paper. We have the following advantages negligible in  $\lambda$  for an efficient adversary  $\mathcal{A}$ :

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{SFPK}}^{\text{ch}}(\lambda) &= \left| \Pr[\text{Exp}_{\mathcal{A}, \text{SFPK}}^{\text{ch}} = 1] - \frac{1}{2} \right| \text{ and} \\ \text{Adv}_{\mathcal{A}, \text{SFPK}}^{\text{suf-cma}}(\lambda) &= \Pr[\text{Exp}_{\mathcal{A}, \text{SFPK}}^{\text{suf-cma}} = 1] \end{aligned}$$

## 2.3. Digital Signatures

Digital signatures (DS) [21] are standard cryptographic primitives defined by three algorithms ( $\text{KeyGen}, \text{Sign}, \text{Verify}$ ). They satisfy *correctness* where with overwhelming probability we have  $1 \leftarrow \text{Verify}(\text{pk}, m, \text{Sign}(\text{sk}, m))$  for  $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\lambda)$ . Their security property ensures that they are resistant to existential forgeries under adversarially chosen messages.

1. This is a slight modification of [11] and we expand on this in Section 6.

## 2.4. Non-Interactive Zero-Knowledge Proofs

For a given relation  $R$ , a language is defined as  $L_R := \{x : \exists w (x, w) \in R\}$ . A zero-knowledge proof system allows a prover to prove knowledge of a witness  $w$  to the relation  $R$ , without leaking any information about it. Formally, it is defined below.

**Definition 4** (Non-Interactive Zero-Knowledge Proof System [24]). A NIZK proof system for a relation  $R$  is a tuple of algorithms (Setup, Prove, Verify, SimSetup, Sim) defined as follows.

$\rho \leftarrow \text{Setup}(\lambda)$  takes as input the security parameter  $\lambda$  and outputs a common reference string  $\rho$ .  
 $\pi \leftarrow \text{Prove}(\rho, x, w)$  takes a common reference string  $\rho$ , statement  $x$  and witness  $w$ , returning a proof  $\pi$ .  
 $\{0, 1\} \leftarrow \text{Verify}(\rho, x, \pi)$  outputs 1 if  $\pi$  is a valid proof with respect to the common reference string  $\rho$  and statement  $x$ , and 0 otherwise.  
 $(\rho', \tau) \leftarrow \text{SimSetup}(\lambda)$  takes as input the security parameter  $\lambda$  and outputs a common reference string  $\rho'$  and a trapdoor  $\tau$ .  
 $\pi' \leftarrow \text{Sim}(\rho', \tau)$  takes a common reference string  $\rho'$ , statement  $x$  and returns a simulated proof  $\pi'$ .

A NIZK proof system is *correct* if for any  $\rho \leftarrow \text{Setup}(\lambda)$  and  $(x, w) \in R$  it follows that with overwhelming probability we have  $1 \leftarrow \text{Verify}(\rho, x, \text{Prove}(\rho, x, w))$ .

*Soundness.* A NIZK proof system is sound if the following advantage is negligible in the security parameter  $\lambda$ .

$$\text{Adv}_{\mathcal{A}, \text{NIZK}}^{\text{sound}}(\lambda) = \Pr \left[ \begin{array}{l} \rho \leftarrow \text{NIZK.Setup}(\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\rho) \\ \text{return NIZK.Verify}(\rho, x, \pi) \\ \wedge x \notin L_R \end{array} \right]$$

*Non-interactive Zero Knowledge.* A NIZK proof system has the zero-knowledge property if the following advantage is negligible in the security parameter  $\lambda$ . The advantage  $\text{Adv}_{\mathcal{A}, \text{NIZK}}^{\text{zk}}(\lambda)$  is defined as:

$$\left| \Pr[\rho \leftarrow \text{NIZK.Setup}(\lambda); \text{return } \mathcal{A}^{\text{NIZK.Prove}(\rho, \cdot, \cdot)}(\rho)] - \Pr[(\rho', \tau) \leftarrow \text{SimSetup}(\lambda); \text{return } \mathcal{A}^{\text{Sim}(\rho', \tau, \cdot)}(\rho')] \right|$$

## 3. Syntax & Security Properties for DGS-VLR

In this section we give the syntax and properties of Dynamic Group Signatures with Verifier-Local Revocation. It is modeled based on the definition of Fully Dynamic Group Signatures [6] but with the requirement that the joining phase can be carried out on a public channel. Furthermore, the model [6] separates out the group manager and tracing authorities; however we consider the case that both aspects are managed by the group manager, a consequence of implementing verifier-local revocation. We adopt standard security properties, namely anonymity, traceability, non-frameability (inspired by [6]), and Membership Privacy (inspired by [7]), with minor adaptations to account for the new functionality of the group manager to revoke users from the scheme.

**Definition 5** (Dynamic Group Signatures with Verifier Local Revocation). A Dynamic Group Signature with Verifier Local Revocation DGS-VLR consists of the following algorithms:

$\text{pp} \leftarrow \text{Setup}(\lambda)$  takes as input the security parameter  $\lambda$  and outputs a set of public parameters  $\text{pp}$ . As part of the public parameters we also initialise an empty data structure  $\text{REG}$  that maps user identities with their long term public keys. The parameter  $\text{pp}$  is an implicit input to every other algorithm.  
 $(\text{gpk}, \text{gmsk}, \text{info}) \leftarrow \text{GKeyGen}()$  generates a group public key  $\text{gpk}$  and a group manager secret key  $\text{gmsk}$ . It also initializes the public group information  $\text{info}$ .  
 $(\text{usk}_{id}, \text{upk}_{id}) \leftarrow \text{RegUser}(id)$ , executed by the user, it samples a private-public key pair  $(\text{usk}_{id}, \text{upk}_{id})$  given the user identity  $id$ . The registry  $\text{REG}$  is updated only if nothing is previously stored for this user, e.g., if  $\text{REG}[id] = \perp$  then  $\text{REG}[id] \leftarrow \text{upk}_{id}$ .  
 $(\text{gmsk}, \text{info}) \leftarrow \text{AddUser}(\text{gmsk}, \text{info}, id)$  executed by the group manager, this algorithm adds a registered user  $id$  to the group, and updates the group manager secret key  $\text{gmsk}$  and public group information  $\text{info}$ .  
 $\text{info} \leftarrow \text{Revoke}(\text{gmsk}, \text{info}, id)$  revokes a group member  $id$ , updating the public group information  $\text{info}$  accordingly.  
 $\Sigma \leftarrow \text{Sign}(\text{info}, \text{usk}_{id}, m)$  run by user  $id$ , it uses the public group information  $\text{info}$ , the user's secret key  $\text{usk}_{id}$  and a message  $m$  to produce a signature  $\Sigma$  on  $m$ .  
 $\{0, 1\} \leftarrow \text{Verify}(\text{gpk}, \text{info}, m, \Sigma)$  checks whether  $(m, \Sigma)$  forms a valid message-signature pair against the group public key and information  $(\text{gpk}, \text{info})$  and a list of revocation tokens  $\text{RL}$ . For ease, we include  $\text{RL}$  in  $\text{info}$ . It outputs a bit  $b \in \{0, 1\}$  with 1 indicating acceptance and 0 rejection.  
 $(id, \pi) \leftarrow \text{Open}(\text{gpk}, \text{gmsk}, \text{info}, m, \Sigma)$  outputs the identity  $id$  of a user along with a proof  $\pi$  tracing the signature  $(m, \Sigma)$  to this user. In the case that  $\text{Open}$  fails to trace, it outputs  $(\perp, \perp)$ .  
 $\{0, 1\} \leftarrow \text{Judge}(\text{gpk}, \text{info}, id, \pi, m, \sigma)$  uses the group public key and information  $\text{gpk}, \text{info}$  to validate a proof of authorship  $\pi$  for some message-signature pair  $(m, \sigma)$  attributing it to a user  $id$ . It outputs a bit  $b \in \{0, 1\}$  with 1 indicating acceptance and 0 rejection.

**Remark 1.** The separation of the join/issue protocol into the  $\text{RegUser}$  and  $\text{AddUser}$  algorithms allows the possibility for the same user to be added to the group multiple times under a single registry. In particular this behaviour is useful for reinstating a revoked user without having to allocate them a new  $id$ , i.e. temporary revocation. Alternatively, enforcing  $\text{AddUser}$  to ignore existing or revoked group members recovers existing models [6], [7].

### 3.1. Correctness and Security Properties

We now state the formal definitions of correctness and the security properties.

**Notation.** The experiments make use of the following additional lists.

- $HU$ : A list of honest registered users, populated by the registration oracle  $HonestU$ . For any  $id \in HU$  we have a stored public key  $REG[id] \neq \perp$ . The adversary can additionally create users outside of this ( $id' \notin HU$ ) and populate  $REG[id']$  directly with some public key.
- $BU$ : A list of corrupt users, whose keys have been compromised by the adversary calling  $RevealU$ . We have that for any  $id \in BU$  implies  $id \in HU$ .
- $AM$ : A list of active group members who are able to sign successfully. An active group member becomes inactive if revoked by  $RevokeU$ . Note  $id \in AM$  implies  $REG[id] \neq \perp$ , but does not imply  $id \in HU$ .
- $Q_{sig}$ : list of queries  $(id, m)$  made as input to  $SignU$ .
- $OBL$ : A list of message-signature pairs that the adversary has been challenged on. The adversary is prohibited from running the Open algorithm (via the oracle  $OpenSig$ ) to identify the user that issued that signature.
- $Chl = \{id_0, id_1\}$ : Stores a pair of challenge users chosen by the adversary in the anonymity, join, and leave experiments.

**Oracles.** We introduce the following oracles that are used in the security experiments.

- $upk_{id} \leftarrow HonestU(id)$  simulates  $RegUser(id)$  to create a private-public key pair  $(usk_{id}, upk_{id})$ , and updates  $HU \leftarrow HU \cup \{id\}$ . Note that the registry is also updated  $REG[id] \leftarrow upk_{id}$ .
- $usk_{id} \leftarrow RevealU(id)$  returns the signing key of a user with the given an identity that is not part of the challenge  $id \in HU \setminus Chl$ . We also update  $BU \leftarrow BU \cup \{id\}$ . Note all  $id \in BU$  implies  $id \in HU$ .
- $CorruptU(id, pk)$  updates  $REG[id] \leftarrow pk$  only if  $REG[id] = \perp$ . This simulates the registration of a public key of a corrupted user (i.e, under the control of the adversary). Note  $REG[id] = \perp$  implies  $id \notin HU$ , but  $REG[id] \neq \perp$  does not imply  $id \in HU$ .
- $info \leftarrow IssueU(id)$  simulates  $AddUser$  on the identity  $id$ , making them an active group member. This adds a new entry to  $info$  associated with the corresponding user even if they were already an active member. The list of active group members is updated  $AM \leftarrow AM \cup \{id\}$ .
- $info \leftarrow ActivateU(id)$  simulates  $AddUser$  only for  $id \notin AM$ . Otherwise, it returns previously stored  $info$ . The list of active group members is updated  $AM \leftarrow AM \cup \{id\}$  as for  $IssueU$ .
- $info \leftarrow RevokeU(id)$  simulates  $Revoke$  to revoke user  $id$  from the group, and updates  $AM \leftarrow AM \setminus \{id\}$ . The user can later be added back to the group by using  $IssueU(id)$  or  $ActivateU(id)$ .
- $\sigma \leftarrow SignU(info, id, m)$  simulates  $Sign$  with  $usk_{id}$  and returns the signature  $\sigma$  for any  $id \in HU \setminus Chl$ . We also update  $Q_{sig} \leftarrow Q_{sig} \cup \{(id, m)\}$ .
- $id \leftarrow OpenSig(m, \sigma)$  simulates  $Open$  and returns the identity  $id$  without a proof, except if  $(m, \sigma)$  is in the opening ban list  $OBL$ .

- $\sigma \leftarrow Chall_{b,anon}(info, m)$  runs  $Sign$  with  $usk_{id_b}$  for the challenge bit  $b \in \{0, 1\}$  and challenge user identities  $Chl = \{id_0, id_1\}$ . It returns the signature  $\sigma$  only if both users have been honestly generated and are active members  $Chl \subseteq HU$  and  $Chl \subseteq AM$ ; otherwise it returns  $\perp$ . Notice that for the anonymity experiment the adversary can corrupt the users and have access to their signing keys, e.g.,  $Chl \subseteq BU$ . The opening ban list is updated  $OBL \leftarrow OBL \cup \{(m, \sigma)\}$ .
- $\sigma \leftarrow Chall_{b,join}(info, m)$  simulates  $Sign$  with  $usk_{id_b}$  for the challenge bit  $b \in \{0, 1\}$  and challenge user identities  $Chl = \{id_0, id_1\}$ . Returns the signature  $\sigma$  only if both users haven't been corrupted  $Chl \subseteq HU$  and  $Chl \cap BU = \emptyset$ . The following list is updated  $OBL \leftarrow OBL \cup \{(m, \sigma)\}$ .
- $Chall_{b,leave}(info, m)$  simulates  $Sign$  with  $usk_{id_b}$  for the challenge bit  $b \in \{0, 1\}$  and challenge user identities  $Chl = \{id_0, id_1\}$ . Returns the signature  $\sigma$ , unless either user is corrupt or has been used to query the signing oracle.

**Correctness.** This ensures that if the protocol steps are followed, for any message  $m$ , with overwhelming probability we have:

- 1) A valid group signature is created by a user if and only if they are an active group member, and
- 2) An honest judge will accept the proof regarding the identity of the signer, given an opening of a valid signature.

Formally, using the experiment in Figure 1 we have

$$1 - \text{negl}(\lambda) \leq \Pr[\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{corr}} = 1].$$

**Anonymity.** A DGS-VLR scheme is anonymous if an adversary cannot determine the identity of a user  $id_b \in \{id_0, id_1\}$  given polynomially many challenge message-signature pairs  $(m, \Sigma)$  under the signing key  $usk_{id_b}$ . This is the case even if the adversary has access to all group members' signing keys and an opening oracle (restricted on the challenge pairs).

**Definition 6** (Anonymity). A dynamic group signature with verifier local revocation scheme is anonymous if the following advantage is negligible in  $\lambda$ , where  $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{anon}}$  is defined in Figure 1.

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{anon}}(\lambda) = \left| \Pr[\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{anon}} = 1] - \frac{1}{2} \right|$$

**Remark 2.** Our anonymity experiment uses two challenges oracles to capture the notion of swapping the order in which the identities  $id_0$  and  $id_1$  are input to the challenge oracle (so the oracle effectively signs for  $1 - b$  instead of  $b$ ). While the experiment is typically defined using a single oracle with malleable input for challenge identities in the literature (e.g. [6], [7]), using distinct oracles leads to both an equivalent security definition and a more direct proof argument for our construction.

**Traceability.** A dynamic group signature scheme is traceable if an adversary cannot create a valid message-signature

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{corr}}(1^\lambda)$ <hr/> 1 : $\text{pp} \leftarrow \$ \text{Setup}(1^\lambda)$<br>2 : $(\text{gpk}, \text{gmsk}, \text{info}) \leftarrow \$ \text{GKeyGen}(\text{pp})$<br>3 : $HU, AM \leftarrow \emptyset$<br>4 : $(m, id) \leftarrow \$ \mathcal{A}(\text{pp}, \text{gpk}, \text{gmsk}, \text{info})^{\text{O1}}$<br>5 : <b>if</b> $id \notin HU \vee id = \perp$ <b>then return</b> 1<br>6 : $\Sigma \leftarrow \text{Sign}(\text{info}, \text{usk}_{id}, m)$<br>7 : $b_1 \leftarrow \text{Verify}(\text{gpk}, \text{info}, m, \Sigma)$<br>8 : <b>if</b> $id \notin AM$ <b>then return</b> $\neg b_1$<br>9 : $(id', \tau) \leftarrow \text{Open}(\text{gpk}, \text{gmsk}, \text{info}, m, \Sigma)$<br>10 : $b_2 \leftarrow \text{Judge}(\text{gpk}, \text{info}, id, \tau, m, \Sigma)$<br>11 : <b>return</b> $b_1 \wedge b_2 \wedge (id = id')$ | $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{frame}}(1^\lambda)$ with $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ <hr/> 1 : $\text{pp} \leftarrow \$ \text{Setup}(1^\lambda)$<br>2 : $(\text{st}, \text{gpk}, \text{gmsk}, \text{info}) \leftarrow \mathcal{A}_0(\text{pp})$<br>3 : <b>if</b> $\text{gpk} = \perp \vee \text{gmsk} = \perp \vee \text{info} = \perp$<br>4 : <b>then return</b> 0<br>5 : $HU, BU, Q_{sig} \leftarrow \emptyset$<br>6 : $(\text{info}, m, \Sigma, id, \tau) \leftarrow \mathcal{A}_1(\text{st})^{\text{O2}}$<br>7 : $b \leftarrow 0$<br>8 : <b>if</b> $id \in HU \setminus BU \wedge (id, m) \notin Q_{sig}$<br>9 : $\wedge \text{Verify}(\text{gpk}, \text{info}, m, \Sigma)$ <b>then</b><br>10 : $b \leftarrow \text{Judge}(\text{gpk}, \text{info}, id, \tau, m, \Sigma)$<br>11 : <b>return</b> $b$                                                                                             |
| $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{anon}}(1^\lambda)$ with $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ <hr/> 1 : $\text{pp} \leftarrow \$ \text{Setup}(1^\lambda)$<br>2 : $(\text{gpk}, \text{gmsk}, \text{info}) \leftarrow \$ \text{GKeyGen}(\text{pp})$<br>3 : $HU, BU, AM, Q_{sig}, OBL, Chl \leftarrow \emptyset$<br>4 : $(\text{st}, id_0, id_1) \leftarrow \$ \mathcal{A}_0(\text{pp}, \text{REG}, \text{gpk}, \text{info})^{\text{O3}, \text{O4}}$<br>5 : $Chl \leftarrow \{id_0, id_1\}$<br>6 : $b \leftarrow \$ \{0, 1\}$<br>7 : $b' \leftarrow \$ \mathcal{A}_1(\text{st})^{\text{O3}, \text{O4}, \text{O5}}$<br>8 : <b>return</b> $b' = b$                                                                                                                                                                                                                             | $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{trace}}(1^\lambda)$ <hr/> 1 : $\text{pp} \leftarrow \$ \text{Setup}(1^\lambda)$<br>2 : $(\text{gpk}, \text{gmsk}, \text{info}) \leftarrow \$ \text{GKeyGen}(\text{pp})$<br>3 : $(\text{info}', m, \Sigma) \leftarrow \$ \mathcal{A}(\text{pp}, \text{gpk}, \text{info})^{\text{O3}, \text{O4}}$<br>4 : $b \leftarrow 0$<br>5 : <b>if</b> $\text{Verify}(\text{gpk}, \text{info}', m, \Sigma)$ <b>then</b><br>6 : $(id, \tau) \leftarrow \text{Open}(\text{gpk}, \text{gmsk}, \text{info}, m, \Sigma)$<br>7 : $b \leftarrow \neg \text{Judge}(\text{gpk}, \text{info}, id, \tau, m, \Sigma)$<br>8 : <b>return</b> $b$                                                                                                                                                                                                                                                                   |
| $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{join}}(1^\lambda)$ with $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ <hr/> 1 : $\text{pp} \leftarrow \$ \text{Setup}(1^\lambda)$<br>2 : $(\text{gpk}, \text{gmsk}, \text{info}) \leftarrow \$ \text{GKeyGen}(\text{pp})$<br>3 : $HU, BU, AM, Q_{sig}, OBL, Chl \leftarrow \emptyset$<br>4 : $(\text{st}, id_0, id_1) \leftarrow \mathcal{A}_0(\text{pp}, \text{gpk}, \text{info})^{\text{O3}, \text{O4}}$<br>5 : $Chl \leftarrow \{id_0, id_1\}; b \leftarrow \$ \{0, 1\}$<br>6 : <b>if</b> $Chl \subset HU \setminus BU$ <b>then</b><br>7 : $(\text{gmsk}, \text{info}) \leftarrow \text{AddUser}(\text{gmsk}, \text{info}, id_b)$<br>8 : $b' \leftarrow \mathcal{A}_1(\text{st})^{\text{O3}, \text{O4}, \text{Chall}_{b, \text{join}}}$<br>9 : <b>return</b> $b' = b$                                                                          | $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{leave}}(1^\lambda)$ with $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ <hr/> 1 : $\text{pp} \leftarrow \$ \text{Setup}(1^\lambda)$<br>2 : $(\text{gpk}, \text{gmsk}, \text{info}) \leftarrow \$ \text{GKeyGen}(\text{pp})$<br>3 : $HU, BU, AM, Q_{sig}, OBL, Chl \leftarrow \emptyset$<br>4 : $(\text{st}, id_0, id_1) \leftarrow \mathcal{A}_0(\text{pp}, \text{gpk}, \text{info})^{\text{O3}}$<br>5 : $Chl \leftarrow \{id_0, id_1\}; b \leftarrow \$ \{0, 1\}$<br>6 : <b>if</b> $Chl \not\subset AM$ <b>then</b> $Chl \leftarrow (\perp, \perp)$<br>7 : <b>if</b> $\forall id \in Chl. id \in HU \setminus BU \wedge (id, \cdot) \notin Q_{sig}$<br>8 : <b>then</b> $\text{info} \leftarrow \text{Revoke}(\text{gmsk}, \text{info}, id_b)$<br>9 : $b' \leftarrow \mathcal{A}_1(\text{st})^{\text{O3}, \text{O4}, \text{Chall}_{1-b, \text{leave}}}$<br>10 : <b>return</b> $b' = b$ |

Figure 1. Group signature security games with oracles  $\text{O1} = \{\text{HonestU}, \text{IssueU}, \text{RevokeU}\}$ ,  $\text{O2} = \{\text{HonestU}, \text{CorruptU}, \text{RevealU}, \text{SignU}\}$ ,  $\text{O3} = \{\text{HonestU}, \text{CorruptU}, \text{RevealU}, \text{ActivateU}, \text{RevokeU}, \text{SignU}, \text{OpenSig}\}$ ,  $\text{O4} = \{\text{IssueU}\}$ , and  $\text{O5} = \{\text{Chall}_{b, \text{anon}}, \text{Chall}_{1-b, \text{anon}}\}$ .

pair  $(m, \Sigma)$  that does not open to a valid identity  $id$  and proof  $\pi$  for an existing group member. Note that our definition slightly departs from existing work (e.g. [6], [7]) as we do not separate the role of tracing authority and group manager.

**Definition 7** (Traceability). A dynamic group signature scheme is traceable if the following advantage is negligible in  $\lambda$ , where  $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{trace}}$  is defined in Figure 1.

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{trace}}(\lambda) = \Pr[\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{trace}} = 1]$$

**Non-Frameability.** A dynamic group signature scheme is non-frameable if an adversary cannot create a valid message-signature pair  $(m, \Sigma)$  that opens to an identity  $id$  of an

honest user for which the adversary does not know the user secret key, even if the adversary can corrupt the group manager.

**Definition 8** (Non-Frameability). A dynamic group signature scheme is non-frameable if the following advantage is negligible in  $\lambda$ , where  $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{frame}}$  is defined in Figure 1.

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{frame}}(\lambda) = \Pr[\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{frame}} = 1]$$

**Membership Privacy.** Introduced by Backes et al. [7], membership privacy captures strong privacy properties for dynamic group signatures. Intuitively, it ensures privacy as groups members are added and revoked. We adopt these

notions for our scheme, and say a DGS-VLR provides membership privacy if it provides both join and leave privacy.

- The scheme provides join privacy if an adversary cannot distinguish which of two chosen honest users  $id_0, id_1$  has been issued (regardless of whether each user is already active).
- The scheme provides leave privacy if an adversary cannot distinguish which of two chosen active honest users  $id_0, id_1$  has been revoked.

In both games the adversary cannot request signatures on either user after the challenge is given, instead they may receive signatures anonymously via a challenge oracle for the activated user  $id_b$  in the join game and the non-revoked user  $id_{1-b}$  in the leave game. In particular, the adversary cannot use the opening oracle on these signatures.

**Definition 9** (Join Privacy). A dynamic group signature with VLR provides join-privacy if the following advantage is negligible in  $\lambda$ , where  $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{join}}$  is defined in Figure 1.

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{join}}(\lambda) = \left| \Pr \left[ \text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{join}} = 1 \right] - \frac{1}{2} \right|$$

**Definition 10** (Leave Privacy). A dynamic group signature with VLR provides leave-privacy if the following advantage is negligible in  $\lambda$ , for  $\text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{leave}}$  defined in Figure 1.

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{leave}}(\lambda) = \left| \Pr \left[ \text{Exp}_{\mathcal{A}, \text{DGS-VLR}}^{\text{leave}} = 1 \right] - \frac{1}{2} \right|$$

**Remark 3.** The consequences of our modeling choices as discussed in Remark 1 must be carefully considered for the membership privacy experiments. For the join game, no problems arise as the change in info is the same regardless of whether  $id_b$  has been issued previously before the challenge is declared. However, for the leave game when the challenge is declared and  $id_b$  is revoked, all group information linked to  $id_b$  is impacted simultaneously. The adversary may link the amount of affected information to the number of times  $id_b$  was issued before the challenge was declared. Thus the experiment must limit each identity to only one issuing in the first phase. This purpose is fulfilled by the *ActivateU* oracle, while the *IssueU* oracle is prohibited for  $\mathcal{A}_0$  in the leave game. Furthermore, the leave game requires that both challenge users are active members, however since one of them is revoked before the guess phase, the  $\text{Chall}_{b, \text{leave}}$  oracle will not be able to check this condition directly. This is addressed by setting both challenge users to  $(\perp, \perp)$  if the check fails before revocation. Then, the oracle will reject considering these users dishonest.

## 4. Generic Construction of DGS-VLR

In this section we detail our construction for DGS-VLR, and illustrate all our algorithms in Figure 2.

The group manager executes Setup to generate the public parameters by running the setup algorithms for an asynchronous remote key generation ARKG and a signature with

flexible public keys SFPK. It instantiates a group by creating keys for a digital signature DS, as well as initialising both a list of group members  $pkl$  and a revocation list RL as empty. These are output as a group information info. The process for building the group can be summarised by the following six steps.

- Users register to be considered for the group.
- Group manager selects group members from the list of users in the registry.
- Group manager creates group signing keys for each group member via ARKG's DerivePK algorithm.
- Users identify their group signing keys using the group information (i.e. via public channels).
- Users re-randomise their obtained signing key and use it to create a signature via SFPK.
- Group manager may revoke a user by publishing a SFPK trapdoor that identifies all possible randomisations of their signing key.

A user  $id$  registers by first generating an ARKG key pair  $(usk, upk)$  and declaring ownership of the public key  $upk$  via the registry  $\text{REG}[id] \leftarrow upk$ . The user is added to the group by the group manager by executing the AddUser algorithm. This algorithm runs our modified ARKG.DerivePK algorithm on  $upk$  to create a new user key  $pk$  (and corresponding derivation information  $cred, \tau_{cred}$ ). The pk-unlinkability property of ARKG ensures that these two keys cannot be linked, even with knowledge of  $cred$ . The keys can however be linked with knowledge of  $\tau_{cred}$ , which is stored by the group manager so that they may later open signatures to  $id$ . Furthermore, the algorithm creates an additional token  $\tau_{pk}$  by executing SFPK.TrapGen. This is stored by the group manager to enable linking in the case that the user is later revoked, by running the revoke algorithm, which appends  $\tau_{pk}$  to the revoke list RL. This is published, and is authenticated with a digital signature  $\sigma_{info}$ , over  $pkl$  and RL, which verifies against the group managers key  $gpk$ .

To create a group signature, a user runs the Sign algorithm, which takes as input the group information info, its long-term user key  $usk$  and the message  $m$ . It begins by parsing  $pkl$  and identifying which  $pk, cred$  correspond to its key  $usk$ . It does this by checking the MACs stored in  $cred$ . Once found, it runs ARKG.DeriveSK to derive  $sk$  that corresponds to the derived public key  $pk$ . It is important to notice that the derived key space to which  $pk$  belongs is carefully chosen to coincide with the key space for the SFPK scheme, thus it can randomise this key using SFPK.ChgPK, to create a key  $pk'$  along with a corresponding key  $sk'$  using SFPK.ChgSK. The message  $m$  is signed with SFPK.Sign using the key  $sk'$ . The class-hiding property of SFPK ensures that the randomised key  $pk'$  cannot be linked to  $pk$  without knowledge of  $\tau_{pk}$ , and since the keys are randomised for every new signature, it implies that signatures also cannot be linked. Finally, the user creates a zero-knowledge proof  $\Pi$  that the key  $pk'$  indeed is the randomisation of a valid key, i.e. a  $pk$  that is in  $pkl$ . More precisely, the relation for

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Setup(<math>\lambda</math>)</b><br><hr/> 1: $gp \leftarrow \text{ARKG.Setup}(\lambda)$<br>2: $crs \leftarrow \text{SFPK.CRSGen}(gp)$<br>3: $\text{REG} \leftarrow \emptyset$<br>4: $pp \leftarrow \{gp, crs, \text{REG}\}$<br>5: <b>return</b> ( $pp$ )                                                                                                                                                                                                                                         | <b>AddUser(<math>gmsk, info, id</math>)</b><br><hr/> 1: <b>parse</b> $info = (pkl, RL, \sigma_{info})$<br>2: <b>parse</b> $gmsk = (ik, okl)$<br>3: $upk \leftarrow \text{REG}[id]$<br>4: $(\hat{pk}, cred, \tau_{cred}) \leftarrow \text{ARKG.DerivePK}(upk)$<br>5: $(pk, \tau_{pk}) \leftarrow \text{SFPK.TrapGen}(\hat{pk})$<br>6: $pkl \leftarrow pkl \cup \{(pk, \hat{pk}, cred)\}$<br>7: $okl \leftarrow okl \cup \{(id, \tau_{pk}, \tau_{cred})\}$<br>8: $\sigma_{info} \leftarrow \text{DS.Sign}(ik, (pkl, RL))$<br>9: $info \leftarrow (pkl, RL, \sigma_{info})$<br>10: $gmsk \leftarrow (ik, okl)$<br>11: <b>return</b> ( $gmsk, info$ )                                                                                                                                                                                                                                                                                                           | <b>Revoke(<math>gmsk, info, id</math>)</b><br><hr/> 1: <b>parse</b> $info = (pkl, RL, \sigma_{info})$<br>2: <b>parse</b> $gmsk = (ik, okl)$<br>3: <b>for</b> $(id', \tau_{pk}, \cdot) \in okl$ <b>do</b><br>4: <b>if</b> $id = id'$ <b>then</b> $RL \leftarrow RL \cup \{\tau_{pk}\}$<br>5: $\sigma_{info} \leftarrow \text{DS.Sign}(ik, (pkl, RL))$<br>6: $info \leftarrow (pkl, RL, \sigma_{info})$<br>7: <b>return</b> $info$                 |
| <b>GKeyGen()</b><br><hr/> 1: $(gpk, ik) \leftarrow \text{DS.KeyGen}(pp)$<br>2: $pkl, RL, okl \leftarrow \emptyset$<br>3: $gmsk \leftarrow (ik, okl)$<br>4: $info \leftarrow (pkl, RL, \perp)$<br>5: <b>return</b> ( $gpk, gmsk, info$ )                                                                                                                                                                                                                                                            | <b>Sign(<math>info, usk, m</math>)</b><br><hr/> 1: <b>parse</b> $info = (pkl, RL, \sigma_{info})$<br>2: $pk, sk \leftarrow \perp$<br>3: <b>if</b> $\text{DS.Verify}(gpk, (pkl, RL), \sigma_{info})$ <b>then</b><br>4: <b>for</b> $(\hat{pk}, cred) \in pkl$ <b>do</b><br>5: $sk \leftarrow \text{ARKG.DeriveSK}(usk, cred)$<br>6: <b>for</b> $\tau_{pk} \in RL$ <b>do</b><br>7: <b>if</b> $\text{SFPK.ChkRep}(\tau_{pk}, \hat{pk})$ <b>then</b><br>8: $sk \leftarrow \perp$<br>9: <b>if</b> $sk \neq \perp$ <b>then</b> $pk \leftarrow \hat{pk}$ <b>break</b><br>10: <b>if</b> $sk = \perp$ <b>then return</b> $(\perp, \perp, \perp)$<br>11: $r \leftarrow \mathbb{Z}_p^*$ ; $pk' \leftarrow \text{SFPK.ChgPK}(pk, r)$<br>12: $sk' \leftarrow \text{SFPK.ChgSK}(sk, r)$<br>13: $\Pi \leftarrow \text{NIZK.Prove}((pk', pkl), pk)$<br>14: $M := m    pk'$ ; $\sigma \leftarrow \text{SFPK.Sign}(sk', M)$<br>15: <b>return</b> $\Sigma = (\sigma, pk', \Pi)$ | <b>Open(<math>gmsk, gpk, info, m, \Sigma</math>)</b><br><hr/> 1: <b>parse</b> $gmsk = (ik, okl)$<br>2: <b>parse</b> $\Sigma = (\sigma, pk', \Pi)$<br>3: <b>if</b> $\text{Verify}(gpk, info, m, \Sigma)$ <b>then</b><br>4: <b>for</b> $(id, \tau_{pk}, \tau_{cred}) \in okl$ <b>do</b><br>5: <b>if</b> $\text{SFPK.ChkRep}(\tau_{pk}, pk')$ <b>then</b><br>6: <b>return</b> $(id, (\tau_{pk}, \tau_{cred}))$<br>7: <b>return</b> $(\perp, \perp)$ |
| <b>RegUser(<math>id</math>)</b><br><hr/> 1: $upk, usk \leftarrow \perp$<br>2: <b>if</b> $\text{REG}[id] = \perp$ <b>then</b><br>3: $(upk, usk) \leftarrow \text{ARKG.KeyGen}(pp)$<br>4: $\text{REG}[id] \leftarrow upk$<br>5: <b>return</b> ( $upk, usk$ )                                                                                                                                                                                                                                         | <b>Judge(<math>gpk, info, id, \tau, m, \Sigma</math>)</b><br><hr/> 1: <b>parse</b> $\tau = (\tau_{pk}, \tau_{cred})$<br>2: <b>parse</b> $\Sigma = (\sigma, pk', \Pi)$<br>3: <b>if</b> $\neg \text{Verify}(gpk, info, m, \Sigma)$<br>4: <b>then return</b> 0<br>5: $upk \leftarrow \text{REG}[id]$<br>6: <b>for</b> $(pk, \hat{pk}, cred) \in pkl$ <b>do</b><br>7: $ainp \leftarrow (upk, \hat{pk}, cred, \tau_{cred})$<br>8: $b_1 \leftarrow \text{ARKG.ChkLink}(ainp)$<br>9: $b_2 \leftarrow \text{SFPK.ChkRep}(\tau_{pk}, pk)$<br>10: $b_3 \leftarrow \text{SFPK.ChkRep}(\tau_{pk}, pk')$<br>11: <b>if</b> $(b_1 \wedge b_2 \wedge b_3)$ <b>then</b><br>12: <b>return</b> 1<br>13: <b>return</b> 0                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Verify(<math>gpk, info, m, \Sigma</math>)</b><br><hr/> 1: <b>parse</b> $info = (pkl, RL, \sigma_{info})$<br>2: <b>parse</b> $\Sigma = (\sigma, pk', \Pi)$<br>3: <b>for</b> $\tau_{pk} \in RL$ <b>do</b><br>4: <b>if</b> $\text{SFPK.ChkRep}(\tau_{pk}, pk')$ <b>then</b><br>5: <b>return</b> 0<br>6: $M := m    pk'$<br>7: <b>return</b> $\text{SFPK.Verify}(pk', M, \sigma)$<br>8: $\wedge \text{DS.Verify}(gpk, (pkl, RL), \sigma_{info})$<br>9: $\wedge \text{NIZK.Verify}((pk', pkl), \Pi)$ |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Figure 2. All algorithms for the group signature DGS-VLR.

the proof  $\Pi$  is given by:

$$\mathcal{R} := \{(pk', pkl), (pk); \exists r \text{ s.t. } \text{ChgPK}(pk, r) = pk' \wedge pk \in pkl\}$$

The group signature  $\Sigma$  is a tuple consisting of the SFPK signature  $\sigma$ , the verification key  $pk'$  and the proof  $\Pi$ .

Verification is a public process which takes as input a candidate signature  $\Sigma$ , a message  $m$  and group information  $info$ . The verifier parses revocation tokens  $\tau_{pk}$  from the revocation list  $RL$ , and for each one, runs  $\text{SFPK.ChkRep}$  against the verification key  $pk'$  that comprises part of  $\Sigma$ . If the algorithm returns 1, then a user's trapdoor has been added to  $RL$  and is thus revoked, and verification fails. Else, the SFPK signature  $\sigma$  is verified against  $pk'$  on  $m$ , and the validity of the proof  $\Pi$  is verified. If all stages are successful, then the algorithm outputs 1, else it returns 0.

To trace a signature, the group manager runs the Open algorithm by checking for which SFPK trapdoor  $\tau_{pk}$  matches

the verification key  $pk'$ . It then outputs the corresponding identity  $id$ . This step can be publicly checked in the Judge algorithm, by verifying a signature, and then checking that the group manager's provided  $\tau_{cred}$  links a user's long-term key  $upk$  to some intermediate key  $pk$  in  $info$ . Then it checks that the SFPK trapdoor  $\tau_{pk}$  is a valid trapdoor for  $pk$  by running  $\text{SFPK.ChkRep}$ , and finally checks that the ephemeral verification key  $pk'$  belongs to the same equivalence class by once more running  $\text{SFPK.ChkRep}$ .

As observed in Remark 1, it is possible to issue multiple public keys to the same group member, while each token  $\tau_{pk}$  only revokes the individual key class  $[pk]$ . This provides the ability to reinstate a previously revoked user by issuing them a new public key.

## 5. Security Analysis

In this section we prove that our construction of DGS-VLR (Figure 2), is correct and satisfies Anonymity, Trace-

ability, Non-frameability, Join Privacy and Leave Privacy.

**Correctness (sketch).** Our scheme being correct intuitively relies on the correctness of the underlying building blocks. Furthermore, we require that the keys generated by ARKG are compatible with the instantiation of SFPK. If this is the case, then ARKG can be used to create valid keys for SFPK, and thus SFPK will not fail to produce an output. Correctness of the NIZK ensures that, since the signer has a valid witness (i.e. their derived secret key and its randomisation), then it can produce a valid proof  $\Pi$ .

**Theorem 1** (Anonymity). *The dynamic group signature scheme DGS-VLR defined in Figure 2 achieves the anonymity security property if DS is EUF-CMA secure, NIZK has the zero-knowledge property and SFPK is both class-hiding and SUF-CMA secure. More precisely, an adversary  $\mathcal{A}$  wins the anonymity game defined in Definition 6 with advantage:*

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{anon}}(\lambda) &\leq \text{Adv}_{\mathcal{A}, \text{NIZK}}^{\text{zk}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda) \\ &\quad + n(n-1) \text{Adv}_{\mathcal{A}, \text{SFPK}}^{\text{suf-cma}}(\lambda) \\ &\quad + n(n-1) \text{Adv}_{\mathcal{A}, \text{SFPK}}^{\text{ch}}(\lambda), \end{aligned}$$

where  $n$  is the size of the group.

*Proof.* Denote by  $\varepsilon_k^{\text{anon}}$  the probability  $|\Pr[\text{Anon}_k] - \frac{1}{2}|$  where  $k \in \{1, \dots, 5\}$ . Notice that

$$\varepsilon_1^{\text{anon}} := \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{anon}}(\lambda).$$

We introduce the following sequence of games, starting from the anonymity experiment.

Anon<sub>1</sub> Original anonymity game.

Anon<sub>2</sub> Modify the *SignU* oracle such that instead of checking  $\text{DS.Verify}(\text{gpk}, \sigma_{\text{info}})$ , check that the value of info given matches some previous output of either *IssueU* or *RevokeU* tracked by a list  $IL$ .

Anon<sub>3</sub> Replace the prover used to generate the NIZK proof in each signature with a simulator.

Anon<sub>4</sub> Extend line 5 as follows. Choose random  $i, j \leftarrow \{1, \dots, n\}$ , if  $(id_0, id_1) \neq (i, j)$  then abort.

Anon<sub>5</sub> In line 7 replace  $\mathcal{A}$ 's challenge oracles  $\text{Chall}_{b, \text{anon}}, \text{Chall}_{1-b, \text{anon}}$  with two copies of  $\text{Chall}_{0, \text{anon}}$ .

We denote by  $\varepsilon_k^{\text{anon}}$  the probability  $|\Pr[\text{Anon}_k] - \frac{1}{2}|$  where  $k \in \{1, \dots, 5\}$ . Notice that

$$\varepsilon_1^{\text{anon}} := \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{anon}}(\lambda).$$

Consider an adversary  $\mathcal{A}$  that wins Anon<sub>1</sub> but not Anon<sub>2</sub>. By the correctness of DS, every  $\text{info} \in IL$  will contain  $\sigma_{\text{info}}$  such that  $\text{DS.Verify}(\text{gpk}, \sigma_{\text{info}}) = 1$  so *SignU* will behave identically in Anon<sub>1</sub> and Anon<sub>2</sub> for such queries. But  $\mathcal{A}$  achieves a different outcome in Anon<sub>2</sub> than in Anon<sub>1</sub> and hence it must make at least one query to *SignU* with some input value  $\text{info} \notin IL$ . Furthermore, in the case that  $\text{DS.Verify}$  rejects some  $\sigma_{\text{info}}$ , the output of *SignU* is simply  $(\perp, \perp, \perp)$  and does not depend on the input  $id$ . In particular, when *SignU* is called via  $\text{Chall}_{b, \text{join}}$  its output does not depend on the challenge bit  $b$ . Therefore  $\mathcal{A}$  must make at

least one query to *SignU* with input  $\text{info} \notin IL$  such that  $\text{DS.Verify}(\text{gpk}, (\text{pk}_l, \text{RL}), \sigma_{\text{info}}) = 1$ .

We construct an adversary  $\mathcal{B}$  against the euf-cma game for DS that simulates Anon<sub>1</sub> for  $\mathcal{A}$  while also storing  $IL$  in its state. In order to simulate *IssueU* and *RevokeU*  $\mathcal{B}$  uses the euf-cma signing oracle. For each query to *SignU*  $\mathcal{B}$  checks whether  $\text{info} \notin IL$  and  $\text{DS.Verify}(\text{gpk}, (\text{pk}_l, \text{RL}), \sigma_{\text{info}}) = 1$ . If such a query is made  $\mathcal{B}$  terminates the simulation of Anon<sub>1</sub> and adds  $\text{info}$  to  $IL$ . Once Anon<sub>1</sub> terminates as above, or otherwise,  $\mathcal{B}$  submits  $IL$  to the euf-cma game, winning in the former case. Recall that this is precisely the case that  $\mathcal{A}$  wins the simulated instance of Anon<sub>1</sub> without winning the corresponding instance of Anon<sub>2</sub>.

$$|\varepsilon_1^{\text{anon}} - \varepsilon_2^{\text{anon}}| \leq \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda)$$

An adversary that can distinguish Anon<sub>2</sub> from Anon<sub>3</sub> can distinguish the simulator from the prover, breaking the zero-knowledge property of NIZK.

$$|\varepsilon_2^{\text{anon}} - \varepsilon_3^{\text{anon}}| \leq \text{Adv}_{\mathcal{A}, \text{NIZK}}^{\text{z-k}}(\lambda)$$

Next we consider Anon<sub>4</sub>. The probability that the adversary chooses the required values for  $i, j$  to avoid Anon<sub>4</sub> being aborted is  $1/n(n-1)$ , so its advantage is adjusted by the same factor.

$$\varepsilon_4^{\text{anon}} = \frac{1}{n(n-1)} \varepsilon_3^{\text{anon}}$$

Consider an adversary  $\mathcal{A}$  against the problem of distinguishing between Anon<sub>4</sub> and Anon<sub>5</sub>. If  $\mathcal{A}$  uses either  $\text{usk}_i$  or  $\text{usk}_j$  to produce a signature against  $\text{info}$  and then submits the result to *OpenSig*, the behaviour of *OpenSig* will be identical in both games. We may therefore assume without loss of generality that  $\mathcal{A}$  does not make such queries. However, if  $\mathcal{A}$  instead submits a signature that depends on some output of either challenge oracle, that is it depends on the underlying key  $\text{pk}$  that the oracle is using, and also if this submission is not in *OBL* and successfully verifies (otherwise *OpenSig* would simply reject the query), then the behaviour of *OpenSig* could depend on which game they are playing. Suppose that  $\mathcal{A}$  does not make such a query, then  $\mathcal{A}$  can be used to construct an adversary  $\mathcal{B}$  against the class-hiding game as follows.

Upon receiving the challenge  $(\text{pp}, (\text{sk}_0, \text{pk}_0), (\text{sk}_1, \text{pk}_1), \text{pk}')$  from the class-hiding game,  $\mathcal{B}$  generates the group keys for Anon<sub>4</sub> by calling  $\text{GKeyGen}(\text{pp})$ . In the choose phase  $\mathcal{B}$  simulates the oracles for  $\mathcal{A}$  according to the normal protocol of DGS-VLR except that on the  $i$ -th and  $j$ -th calls to *HonestU*,  $\mathcal{B}$  takes  $(\text{usk}_i, \text{upk}_i) \leftarrow (\text{sk}_0, \text{pk}_0)$  and  $(\text{usk}_j, \text{upk}_j) \leftarrow (\text{sk}_1, \text{pk}_1)$  respectively. These calls also fix the salt generated in  $\text{ARKG.DerivePK}$  such that  $ck = 1$ , ensuring both key pairs remain unchanged in this step. As an additional change,  $\mathcal{B}$  simulates  $\text{Chall}_{0, \text{anon}}$  using  $\text{sk}_0$  as it would  $\text{SignU}(\cdot, id_0, \cdot)$  and  $\text{Chall}_{b, \text{anon}}$  using  $\text{sk}'$  to sign via its oracles  $\mathcal{O}_{\text{chSign1}}, \mathcal{O}_{\text{chSign2}}$ . Note that it does not matter that  $\mathcal{B}$  cannot simulate *OpenSig* for signatures against keys in

the classes  $[pk_0]$  or  $[pk_1]$  that it did not create itself, as by assumption  $\mathcal{A}$  never makes such queries.

Then  $\mathcal{B}$  sends this game instance to  $\mathcal{A}$ , which outputs a bit  $b' \in \{0, 1\}$  with  $b' = 0$  indicating that the given instance represents  $\text{Anon}_5$  and  $b' = 1$  indicating  $\text{Anon}_4$ . Now if  $b = 0$  then we have  $sk' = sk_0$  matching the protocol of  $\text{Anon}_5$ , whereas if  $b = 1$  then  $sk' = sk_1$  matching the protocol of  $\text{Anon}_4$ . Thus by forwarding the bit  $b'$  to its own game,  $\mathcal{B}$  wins in the case that  $\mathcal{A}$  was able to identify the game instance correctly. Now suppose instead that  $\mathcal{A}$  makes some query to  $\text{OpenSig}$  depending on the underlying key  $pk$  of one of the challenge oracles. Then  $\mathcal{A}$  can be used to construct another adversary  $\mathcal{C}$ , this time against the suf-cma game for SFPK.

Upon receiving the challenge  $(pp, pk, \tau)$  from the suf-cma game,  $\mathcal{C}$  generates the group keys for  $\text{Anon}_4$  by calling  $\text{GKeyGen}(pp)$ . In the choose phase  $\mathcal{C}$  simulates the oracles for  $\mathcal{A}$  according to the normal protocol of DGS-VLR except that for  $\text{Chall}_{b, \text{anon}}$  and  $\text{Chall}_{1-b, \text{anon}}$  it uses its signing oracle  $\mathcal{O}_{\text{unfSign2}}$  with a random salt  $r$ , ignoring the user keys  $upk_i, upk_j$ .

When  $\mathcal{A}$  makes its query to either  $\text{Chall}_{b, \text{anon}}$  or  $\text{Chall}_{1-b, \text{anon}}$ ,  $\mathcal{C}$  terminates the simulation of  $\text{Anon}_4$ , extracts the SFPK message-signature pair from the query and submits this to its own game. Because  $\text{OpenSig}$  would not normally reject this query, it must be distinct from any of the outputs of  $\text{Chall}_{b, \text{anon}}$  or  $\text{Chall}_{1-b, \text{anon}}$  i.e. from the outputs of  $\mathcal{O}_{\text{Sign2}}$ . Furthermore,  $\text{Verify}$  must accept the query and in particular  $\text{SFPK.Verify}$  must accept the extracted message-signature pair so that  $\mathcal{C}$  wins the suf-cma game.

Combining the above results from both possible cases for the behaviour of  $\mathcal{A}$  arrives at the following.

$$|\varepsilon_4^{\text{anon}} - \varepsilon_5^{\text{anon}}| \leq \text{Adv}_{\mathcal{B}, \text{SFPK}}^{\text{ch}}(\lambda) + \text{Adv}_{\mathcal{C}, \text{SFPK}}^{\text{suf-cma}}(\lambda)$$

Finally, in  $\text{Anon}_5$  both signatures are sampled identically so it is statistically impossible to distinguish them. Furthermore, the NIZK simulator does not depend on the value of  $b$  since it is not given a witness. Therefore  $\mathcal{A}$  has a probability of exactly  $\frac{1}{2}$  of winning  $\text{Anon}_5$ .

$$\varepsilon_5^{\text{anon}} = 0$$

Now tracing back the changes in advantage recovers the theorem statement.  $\square$

**Theorem 2** (Traceability). *The dynamic group signature scheme DGS-VLR defined in Figure 2 achieves the traceability security property if NIZK is sound and DS is euf-cma secure. That is, an adversary  $\mathcal{A}$  wins the traceability game defined in Definition 7 with advantage:*

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{trace}}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{NIZK}}^{\text{sound}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda).$$

*Proof.* We consider the following experiments:

Trace<sub>1</sub> Original traceability game.

Trace<sub>2</sub> Add the requirement that  $\text{info}^* \in IL$ , where  $IL$  is the list of all values for info output on some adversarial query to  $\text{IssueU}$ ,  $\text{ActivateU}$  or  $\text{RevokeU}$  oracles.

We start by defining  $\varepsilon_1^{\text{trace}} := \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{trace}}(\lambda)$ . Consider an adversary  $\mathcal{A}$  that wins Trace<sub>1</sub> but not Trace<sub>2</sub>. Then  $\mathcal{A}$  must output some  $\text{info}^* := (pkl^*, \text{RL}^*, \sigma_{\text{info}^*}) \notin IL$  such that  $\text{DS.Verify}(\text{gpk}, (pkl^*, \text{RL}^*), \sigma_{\text{info}^*})$  accepts. Consider further an adversary  $\mathcal{B}$  against the euf-cma game for DS that simulates Trace<sub>1</sub> for  $\mathcal{A}$  using its  $\text{DS.Sign}$  oracle to generate each  $\sigma_{\text{info}}$ . By extracting  $\sigma_{\text{info}}$  from each  $\text{info} \in IL$  and submitting this list along with  $\sigma_{\text{info}^*}$ ,  $\mathcal{B}$  wins its own game.

$$|\varepsilon_1^{\text{trace}} - \varepsilon_2^{\text{trace}}| \leq \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda)$$

We now show that an adversary  $\mathcal{A}$  against Trace<sub>2</sub> may be used to construct an adversary  $\mathcal{B}$  against the soundness of NIZK. This adversary first executes the trace game as described in Figure 1 against  $\mathcal{A}$ . After some time, it receives  $\text{info}^* := (pkl^*, \text{RL}^*, \sigma_{\text{info}^*})$  and a message-signature pair  $(m^*, \Sigma^*)$  with  $\Sigma^* = (pk^*, \sigma^*, \Pi^*)$  from the adversary. Then  $\mathcal{B}$  simply extracts the proof  $\Pi^*$  and submits it to the soundness game for NIZK. We show that if  $\mathcal{A}$  wins the traceability game, then  $\mathcal{B}$  wins the soundness game.

If  $\mathcal{A}$  wins its experiment, then the adversary output  $(m^*, \Sigma^*)$  must open to some  $(id, (\tau_{pk}, \tau_{\text{cred}}))$  where  $\text{Judge}(\text{gpk}, \text{info}, \text{REG}, id, (\tau_{pk}, \tau_{\text{cred}}), m^*, \Sigma^*) = 0$ . Furthermore, we must have  $\text{Verify}(\text{gpk}, \text{info}^*, m^*, \Sigma^*) = 1$  and in particular the final condition that the proof  $\Pi^*$  verifies in line 9 of  $\text{Verify}$  must hold. Then  $\mathcal{B}$  wins except if the relation  $\exists pk \in pkl^* : pk^* \in [pk]$  holds. Now since  $\text{info}^* \in IL$  and elements of  $pkl$  are never removed, this  $pk$  must lie in  $pkl$ . But then the condition  $\text{SFPK.ChkRep}(\tau_{pk}, pk^*) = 1$  on line 5 of  $\text{Open}$  must be met on some iteration  $i$  of the **for** loop containing it during  $\mathcal{A}$ 's experiment. We now argue this cannot be true.

Suppose that  $\text{SFPK.ChkRep}(\tau_{pk_i}, pk^*) = 1$  for  $i \in \{1, \dots, n\}$ . Then line 6 returns  $\tau_{pk} = \tau_{pk_i}$ , causing the condition  $\text{SFPK.ChkRep}(\tau_{pk}, pk^*) = 1$  on line 9 of  $\text{Judge}$  to hold. Furthermore, from line 4 of  $\text{Open}$  it can be seen that  $(id, \tau_{pk}, \tau_{\text{cred}}) \in okl$ . As  $pkl$  and  $okl$  are only ever changed on lines 6 and 7 of  $\text{AddUser}$  respectively, there must also exist some corresponding  $(pk, \text{cred}) \in pkl$ . Now lines 4 and 5 of  $\text{AddUser}$  show that by the correctness of ARKG and the compatibility of ARKG with SFPK the condition  $\text{ARKG.ChkLink}(\text{REG}[id], pk, \text{cred}, \tau_{\text{cred}}) = 1$  on line 7 of  $\text{Judge}$  holds. Similarly, line 5 of  $\text{AddUser}$  together with correctness of SFPK implies that the condition  $\text{SFPK.ChkRep}(\tau_{pk}, pk) = 1$  on line 8 of  $\text{Judge}$  holds. But then with all conditions on lines 7 to 9 of  $\text{Judge}$  fulfilled, on line 11 it returns 1 thus contradicting that  $\mathcal{A}$  won.

$$\varepsilon_2^{\text{trace}} \leq \text{Adv}_{\mathcal{B}, \text{NIZK}}^{\text{sound}}(\lambda)$$

Now tracing back the changes in advantage recovers the theorem statement.  $\square$

**Theorem 3** (Non-Frameability). *The dynamic group signature scheme DGS-VLR defined in Figure 2 achieves the non-frameability security property if SFPK is euf-cma secure and ARKG has sk-security. That is, an adversary  $\mathcal{A}$*

wins the non-frameability game defined in Definition 8 with advantage

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{frame}}(\lambda) \leq n \cdot (\text{Adv}_{\mathcal{A}, \text{SFPK}}^{\text{euf-cma}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{sks}}(\lambda)).$$

*Proof.* We introduce the following sequence of games.

Frame<sub>1</sub> Original non-frameability game.

Frame<sub>2</sub> Extend line 6 as follows. Choose random  $i \leftarrow \{1, \dots, n\}$ , if  $id \neq i$  then abort.

Frame<sub>3</sub> We replace  $\text{ARKG.ChkLink}(\text{upk}, \text{pk}, \text{cred}, \tau_{\text{cred}})$  on line 5 of Judge with  $\text{ARKG.ChkKeys}(\text{pp}, \text{sk}, \text{pk})$  where  $\text{sk} \leftarrow \text{ARKG.DeriveSK}(\text{pp}, \text{usk}_i, \text{cred})$ .

Frame<sub>4</sub> In line 6 the adversary also outputs a secret key  $\text{sk}^*$  alongside a message and signature, i.e. it returns  $(\text{sk}^*, m, \sigma)$ . Furthermore, Judge gains the additional check  $\text{ARKG.ChkKeys}(\text{pp}, \text{sk}^*, \text{pk})$  for the submitted key.

Frame<sub>5</sub> User  $i$  now also calls  $\text{SFPK.TrapGen}(\text{upk}_i)$  when generating keys, obtaining a trapdoor  $\tau_i$  and replacing  $\text{upk}_i$  with  $\text{pk}_i$ . The adversary receives  $\tau_i$  and Judge now replaces the two  $\text{SFPK.ChkRep}$  checks on lines 6 and 7 with the single check  $\text{SFPK.ChkRep}(\tau_i, \text{pk}')$ . Additionally, on line 5 we further replace  $\text{ARKG.ChkKeys}(\text{pp}, \text{sk}, \text{pk})$  with  $\text{ARKG.ChkKeys}(\text{pp}, \text{usk}_i, \text{pk})$ . The value of  $\text{cred}$  is ignored.

Frame<sub>6</sub> User  $i$  now uses  $\text{SFPK.TrapGen}$  immediately instead of calling  $\text{ARKG.KeyGen}$  and  $\text{SFPK.TrapGen}$  sequentially, effectively replacing  $\text{ARKG.KeyGen}$  with  $\text{SFPK.KeyGen}$ .

We denote by  $\varepsilon_k^{\text{frame}}$  the probability  $\Pr[\text{Frame}_k]$  where  $k \in \{1, \dots, 6\}$ , where

$$\varepsilon_1^{\text{frame}} := \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{frame}}(\lambda).$$

The probability that the adversary chooses the required valued for  $i$  to avoid Frame<sub>2</sub> being aborted is  $1/n$ , so its advantage is adjusted by the same factor.

$$\varepsilon_2^{\text{frame}} = \frac{1}{n} \varepsilon_1^{\text{frame}}$$

In Frame<sub>2</sub> if Judge accepts then in particular we must have that  $\text{ARKG.ChkLink}(\text{upk}, \text{pk}, \text{cred}, \tau_{\text{cred}})$  accepts. Hence by the linking soundness property  $\text{ARKG.ChkKeys}(\text{pp}, \text{sk}, \text{pk})$  accepts, so Judge will also accept in Frame<sub>3</sub>. If it did not, the adversary would be able to win the Linking Soundness experiment, defined in Definition 2, by submitting the tuple  $(\text{pk}, \text{cred}, \tau_{\text{cred}})$  on line two of the definition. The embedding of the challenge parameters is straight forward; all ARKG related parameters are inherited from the Linking Soundness experiment and instantiated in the DGS-VLR, and all additional parameters (such as for SFPK) are created locally by the soundness adversary. Therefore we conclude:

$$|\varepsilon_2^{\text{frame}} - \varepsilon_3^{\text{frame}}| \leq \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{l-sound}}(\lambda)$$

Consider the following two mutually exclusive classifications for the adversary's behaviour in Frame<sub>3</sub>. One possibility is that  $\mathcal{A}$  views  $\text{ARKG.DerivePK}$  as a pseudo-random function with respect to the relationship between  $\hat{\text{pk}}$  and

$\text{cred}$ . In this case  $\mathcal{A}$  does not distinguish the distribution of  $\hat{\text{pk}}$  under any choice of  $\text{cred}$  from that of  $\text{upk}$  and hence it would perform identically in Frame<sub>5</sub>. The alternative is that  $\mathcal{A}$  exploits  $\text{ARKG.DerivePK}$  by choosing  $\text{cred}$  to herd the output for  $\hat{\text{pk}}$  to a particular value, say to match some known key pair  $(\text{sk}^*, \text{pk}^*) \leftarrow \text{ARKG.KeyGen}$ . Then  $\text{sk}^*$  may be extracted along with the usual forgery  $m, \Sigma$  to win Frame<sub>4</sub>. The overall advantage for Frame<sub>3</sub> is therefore bounded according to the total advantage summed across the two branching games Frame<sub>4</sub> and Frame<sub>5</sub>.

$$\varepsilon_3^{\text{frame}} \leq \varepsilon_4^{\text{frame}} + \varepsilon_5^{\text{frame}}$$

Let's focus on the latter case and consider such an adversary  $\mathcal{A}$  against Frame<sub>4</sub>. An adversary  $\mathcal{B}$  against the sk-security game may be constructed from  $\mathcal{A}$  as follows.

Upon receiving the challenge  $(\text{sk}, \text{pk})$  from the sk-security game,  $\mathcal{B}$  forwards the public parameters  $\text{pp}$  to  $\mathcal{A}_0$ . In the play phase  $\mathcal{B}$  simulates the oracles for  $\mathcal{A}_1$  according to the normal protocol of DGS-VLR except that on the  $i$ -th call to *HonestU*,  $\mathcal{B}$  takes  $(\text{usk}_i, \text{upk}_i) \leftarrow (\text{sk}, \text{pk})$ .

At the end of the play phase  $\mathcal{A}_1$  outputs a guess  $\text{sk}^*$ . In the case that Judge accepts  $\mathcal{A}$ 's guess,  $\mathcal{B}$  additionally extracts  $(\text{pk}^*, \text{cred}^*)$  from the accepting iteration of the for loop in Judge. By forwarding  $(\text{sk}^*, \text{pk}^*, \text{cred}^*)$  to the sk-security game,  $\mathcal{B}$  is also able to win, yielding:

$$\varepsilon_4^{\text{frame}} \leq \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{sks}}(\lambda).$$

Now return to the first case and consider an adversary against Frame<sub>5</sub>. Given that the ARKG keys are compatible with the SFPK instantiation (as required by correctness), the distribution of each  $(\text{pk}_{id}, \text{usk}_{id})$  in Frame<sub>5</sub> is identical to the distribution of each  $(\text{upk}_{id}, \text{usk}_{id})$  in Frame<sub>6</sub> making the two games indistinguishable.

$$\varepsilon_5^{\text{frame}} = \varepsilon_6^{\text{frame}}$$

Now let  $\mathcal{A}$  be an adversary against Frame<sub>6</sub>, we construct  $\mathcal{B}$  against the euf-cma experiment for SFPK as follows.

Upon receiving the challenge  $(\cdot, \text{pk}, \tau)$  from the euf-cma experiment,  $\mathcal{B}$  simulates Frame<sub>6</sub> for  $\mathcal{A}$  as normal except it replaces the key generation in the  $i$ -th call to *HonestU* with the challenge  $(\cdot, \text{pk}, \tau)$ . It simulates signing for user  $i$  using its oracles  $\mathcal{O}_{\text{unfSign1}}, \mathcal{O}_{\text{unfSign2}}$  with uniformly random  $r$ . When  $\mathcal{A}$  outputs its forgery  $(m^*, \sigma^*, id^*, \tau^*)$ ,  $\mathcal{B}$  submits  $(m^*, \sigma^*)$  to its own experiment and wins in the case that  $\mathcal{A}$  wins Frame<sub>6</sub>.

$$\varepsilon_6^{\text{frame}} \leq \text{Adv}_{\mathcal{A}, \text{SFPK}}^{\text{euf-cma}}(\lambda)$$

Now tracing back the changes in advantage recovers the theorem statement.  $\square$

**Theorem 4 (Join Privacy).** *The dynamic group signature scheme DGS-VLR defined in Figure 2 achieves the join-privacy security property if ARKG has pk-unlinkability and DS is euf-cma secure. That is, an adversary  $\mathcal{A}$  wins the join-privacy game defined in Definition 9 with advantage:*

$$\text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{join}}(\lambda) \leq n \cdot \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda) + \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda).$$

*Proof.* Let us consider the series of games be defined as follows:

Join<sub>1</sub> Original join-privacy game.

Join<sub>2</sub> Modify the *SignU* oracle such that instead of checking  $\text{DS.Verify}(\text{gpk}, \sigma_{\text{info}})$ , check that the value of info given matches some previous output of either *IssueU* or *RevokeU* tracked by a list  $IL$ .

Join<sub>3</sub> Extend line 5 as follows. Choose random  $i \leftarrow \{1, \dots, n\}$ , if  $id_b \neq i$  then abort.

Join<sub>4</sub> When *AddUser* is called with  $id_b$  on line 7, instead of running  $\text{ARKG.DerivePK}(\text{upk})$  a fresh pair of user keys  $(pk^*, sk^*)$  is generated using  $\text{ARKG.KeyGen}$  along with a random credential  $\text{cred}^*$ . Additionally, in line 8 the challenge oracle  $\text{Chall}_{b, \text{join}}$  is changed to use  $sk^*$  instead of the algorithm  $\text{ARKG.DeriveSK}(sk[id_b])$ .

We denote by  $\varepsilon_k^{\text{join}}$  the probability  $|\Pr[\text{Join}_k] - \frac{1}{2}|$  where  $k \in [4]$ , and have

$$\varepsilon_1^{\text{join}} := \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{join}}(\lambda).$$

Consider an adversary  $\mathcal{A}$  that wins Join<sub>1</sub> but not Join<sub>2</sub>. By the correctness of DS, every  $\text{info} \in IL$  will contain  $\sigma_{\text{info}}$  such that  $\text{DS.Verify}(\text{gpk}, \sigma_{\text{info}}) = 1$  so *SignU* will behave identically in Join<sub>1</sub> and Join<sub>2</sub> for such queries. But  $\mathcal{A}$  achieves a different outcome in Join<sub>2</sub> than in Join<sub>1</sub> and hence it must make at least one query to *SignU* with some input value  $\text{info} \notin IL$ . Furthermore, in the case that DS.Verify rejects some  $\sigma_{\text{info}}$ , the output of *SignU* is simply  $(\perp, \perp, \perp)$  and does not depend on the input  $id$ . In particular, when *SignU* is called via  $\text{Chall}_{b, \text{join}}$  its output does not depend on the challenge bit  $b$  and hence  $\mathcal{A}$  learns nothing. Therefore  $\mathcal{A}$  must make at least one query to *SignU* with input  $\text{info} \notin IL$  such that  $\text{DS.Verify}(\text{gpk}, (pk_l, \text{RL}), \sigma_{\text{info}}) = 1$ .

We construct an adversary  $\mathcal{B}$  against the euf-cma game for DS that simulates Join<sub>1</sub> for  $\mathcal{A}$  while also storing  $IL$  in its state. To simulate *IssueU* and *RevokeU*  $\mathcal{B}$  uses the euf-cma signing oracle. For each query to *SignU*  $\mathcal{B}$  checks whether  $\text{info} \notin IL$  and  $\text{DS.Verify}(\text{gpk}, (pk_l, \text{RL}), \sigma_{\text{info}}) = 1$ . If such a query is made  $\mathcal{B}$  terminates the simulation of Join<sub>1</sub> and adds  $\text{info}$  to  $IL$ . Once Join<sub>1</sub> terminates as above, or otherwise,  $\mathcal{B}$  submits  $IL$  to the euf-cma game, winning in the former case. This is precisely the case that  $\mathcal{A}$  wins the simulated instance of Join<sub>1</sub> without winning the corresponding instance of Join<sub>2</sub>.

$$|\varepsilon_1^{\text{join}} - \varepsilon_2^{\text{join}}| \leq \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda)$$

The probability that the adversary chooses the required value for  $i$  to avoid Join<sub>3</sub> being aborted is  $1/n$ , so its advantage is adjusted by the same factor.

$$\varepsilon_3^{\text{join}} = \frac{1}{n} \varepsilon_2^{\text{join}}$$

An adversary  $\mathcal{A}$  that can distinguish Join<sub>3</sub> from Join<sub>4</sub> can be used to construct an adversary  $\mathcal{B}$  against the pk-unlinkability game as follows.

Upon receiving the challenge  $(pp, pk)$  from the pk-unlinkability game,  $\mathcal{B}$  generates the group keys for the anonymity game with  $\text{GKeyGen}(pp)$ . In the choose phase

$\mathcal{B}$  simulates the oracles for  $\mathcal{A}_0$  according to the normal protocol of DGS-VLR except that on the  $i$ -th call to *HonestU*,  $\mathcal{B}$  returns  $(i, pk)$ . After the choose phase  $\mathcal{B}$  generates keys  $(sk_b, pk_b, \text{cred})$  from the pk-unlink oracle  $\mathcal{O}_{pku}$ , generates a trapdoor  $(pk^*, \tau_{pk^*}) \leftarrow \text{SFPK.TrapGen}(pk_b)$  and sets  $pk_l \leftarrow pk_l \cup \{(pk^*, \text{cred})\}$ ,  $ok_l \leftarrow ok_l \cup \{(i, \tau_{pk^*}, \perp)\}$  unless  $\mathcal{A}_0$  has invoked *RevealU*( $i$ ). Then  $\mathcal{B}$  generates a signature  $\sigma_{\text{info}} \leftarrow \text{DS.Sign}(ik, (pk_l, \text{RL}))$  and returns  $(pk_l, \text{RL}, \sigma_{\text{info}})$  to  $\mathcal{A}$ . Additionally  $\text{Chall}_{b, \text{join}}$  is changed to use  $sk_b$  instead of  $\text{ARKG.DeriveSK}(sk[id_b])$ .

Then  $\mathcal{B}$  sends this game instance to  $\mathcal{A}$ , which outputs a bit  $b' \in \{0, 1\}$  with  $b' = 0$  indicating that the given instance represents Join<sub>3</sub> and  $b' = 1$  indicating Join<sub>4</sub>. Now if  $b = 0$  then the pk-unlink oracle  $\mathcal{O}_{pku}$  provides keys derived from  $pk$  matching the protocol of Join<sub>3</sub>, whereas if  $b = 1$  then  $\mathcal{O}_{pku}$  instead provides freshly generated keys matching the protocol of Join<sub>4</sub>. Thus by forwarding the bit  $b'$  to its own game,  $\mathcal{B}$  wins in the case that  $\mathcal{A}$  was able to identify the game instance correctly.

$$|\varepsilon_4^{\text{join}} - \varepsilon_3^{\text{join}}| \leq \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda)$$

In the guess phase of Join<sub>4</sub>, the value of info does not depend on  $id_b$  since neither  $pk^*$  nor  $\text{cred}^*$  depends on  $id_b$ . Additionally, the functionality of the challenge oracle  $\text{Chall}_{b, \text{join}}$  does not depend on  $id_b$  since it uses  $sk^*$  to sign. Therefore  $\mathcal{A}$  has a probability of exactly  $\frac{1}{2}$  of winning Join<sub>4</sub>.

$$\varepsilon_4^{\text{join}} = 0$$

Now tracing back the changes in advantage recovers the theorem statement.  $\square$

**Theorem 5** (Leave Privacy). *The dynamic group signature scheme DGS-VLR defined in Figure 2 achieves the leave-privacy security property if ARKG has pk-unlinkability and DS is euf-cma secure. That is, an adversary  $\mathcal{A}$  wins the leave-privacy game defined in Definition 10 with advantage*

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{leave}}(\lambda) &\leq 2n(n-1) \cdot \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda) \\ &\quad + \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda). \end{aligned}$$

*Proof.* Let the series of games be defined as follows:

Leave<sub>1</sub> Original leave-privacy game.

Leave<sub>2</sub> Modify the *SignU* oracle such that instead of checking  $\text{DS.Verify}(\text{gpk}, \sigma_{\text{info}})$ , check that the value of info given matches some previous output of either *IssueU* or *RevokeU* tracked by a list  $IL$ .

Leave<sub>3</sub> Extend line 5 as follows. Choose random  $i, j \leftarrow \{1, \dots, n\}$ , if  $(id_0, id_1) \neq (i, j)$  then abort.

Leave<sub>4</sub> In the choose phase for  $\mathcal{A}_0$ , modify *ActivateU* so that on input  $i$ , the  $\text{ARKG.DerivePK}$  call during the *AddUser* step is replaced with a fresh key generation  $(pk^*, sk^*) \leftarrow \text{ARKG.KeyGen}$  and a random credential  $\text{cred}^*$ . Additionally, for queries with user identity  $i$  the *SignU* oracle uses  $sk^*$  directly in place of the derived key  $sk' \leftarrow \text{ARKG.DeriveSK}(sk_i)$ .

Leave<sub>4</sub> Perform the same change made in the previous game but now for both identity  $i$  and identity  $j$ .

We denote by  $\varepsilon_k^{\text{leave}}$  the probability  $|\Pr[\text{Leave}_k] - \frac{1}{2}|$  where  $k \in [5]$  and

$$\varepsilon_1^{\text{leave}} := \text{Adv}_{\mathcal{A}, \text{DGS-VLR}}^{\text{leave}}(\lambda).$$

Consider an adversary  $\mathcal{A}$  that wins  $\text{Leave}_1$  but not  $\text{Leave}_2$ . By the correctness of DS, every  $\text{info} \in IL$  will contain  $\sigma_{\text{info}}$  such that  $\text{DS.Verify}(\text{gpk}, \sigma_{\text{info}}) = 1$  so  $\text{SignU}$  will behave identically in  $\text{Leave}_1$  and  $\text{Leave}_2$  for such queries. But  $\mathcal{A}$  achieves a different outcome in  $\text{Leave}_2$  than in  $\text{Leave}_1$  and hence it must make at least one query to  $\text{SignU}$  with some input value  $\text{info} \notin IL$ . Furthermore, in the case that  $\text{DS.Verify}$  rejects some  $\sigma_{\text{info}}$ , the output of  $\text{SignU}$  is simply  $(\perp, \perp, \perp)t$  and does not depend on the input  $\text{id}$ . In particular, when  $\text{SignU}$  is called via  $\text{Chall}_{b, \text{join}}$  its output does not depend on the challenge bit  $b$  and hence  $\mathcal{A}$  learns nothing. Therefore  $\mathcal{A}$  must make at least one query to  $\text{SignU}$  with input  $\text{info} \notin IL$  such that  $\text{DS.Verify}(\text{gpk}, (\text{pk}, \text{RL}), \sigma_{\text{info}}) = 1$ .

We construct an adversary  $\mathcal{B}$  against the euf-cma game for DS that simulates  $\text{Leave}_1$  for  $\mathcal{A}$  while also storing  $IL$  in its state. In order to simulate  $\text{IssueU}$  and  $\text{RevokeU}$   $\mathcal{B}$  uses the euf-cma signing oracle. For each query to  $\text{SignU}$   $\mathcal{B}$  checks whether  $\text{info} \notin IL$  and  $\text{DS.Verify}(\text{gpk}, (\text{pk}, \text{RL}), \sigma_{\text{info}}) = 1$ . If such a query is made  $\mathcal{B}$  terminates the simulation of  $\text{Leave}_1$  and adds  $\text{info}$  to  $IL$ . Once  $\text{Leave}_1$  terminates as above, or otherwise,  $\mathcal{B}$  submits  $IL$  to the euf-cma game, winning in the former case. Recall that this is precisely the case that  $\mathcal{A}$  wins the simulated instance of  $\text{Leave}_1$  without winning the corresponding instance of  $\text{Leave}_2$ .

$$|\varepsilon_1^{\text{leave}} - \varepsilon_2^{\text{leave}}| \leq \text{Adv}_{\mathcal{A}, \text{DS}}^{\text{euf-cma}}(\lambda)$$

The probability that the adversary chooses the required values for  $i, j$  to avoid  $\text{Leave}_3$  being aborted is  $1/n(n-1)$ , so its advantage is adjusted by the same factor.

$$\varepsilon_3^{\text{leave}} = \frac{1}{n(n-1)} \varepsilon_2^{\text{leave}}$$

An adversary  $\mathcal{A}$  that can distinguish  $\text{Leave}_3$  from  $\text{Leave}_4$  can be used to construct an adversary  $\mathcal{B}$  against the pk-unlinkability game as follows.

Upon receiving the challenge  $(\text{pp}, \text{pk})$  from the pk-unlinkability game,  $\mathcal{B}$  generates the group keys for  $\text{GKeyGen}(\text{pp})$ . In the choose phase  $\mathcal{B}$  simulates the oracles for  $\mathcal{A}_0$  according to the normal protocol of DGS-VLR except that on the  $i$ -th call to  $\text{HonestU}$ ,  $\mathcal{B}$  returns  $(i, \text{pk})$ . Furthermore, when  $\text{ActivateU}$  is called on user  $i$   $\mathcal{B}$  replaces the  $\text{ARKG.DerivePK}$  step in  $\text{DGS-VLR.AddUser}$  and instead generates keys  $(\text{sk}_b, \text{pk}_b, \text{cred})$  from the pk-unlink oracle  $\mathcal{O}_{\text{pku}}$ . Additionally the challenge oracle  $\text{Chall}_{1-\tilde{b}, \text{leave}}$  is changed to use  $\text{sk}_b$  instead of  $\text{ARKG.DeriveSK}(\text{sk}[i])$  in the case that  $\tilde{b} = 1 - b$  is chosen in line 5 of the leave game.

Then  $\mathcal{B}$  sends this game instance to  $\mathcal{A}$ , which outputs a bit  $b' \in \{0, 1\}$  with  $b' = 0$  indicating that the given instance represents  $\text{Leave}_3$  and  $b' = 1$  indicating  $\text{Leave}_4$ . Now if  $b = 0$  then the pk-unlink oracle  $\mathcal{O}_{\text{pku}}$  provides keys derived from pk matching the protocol of  $\text{Leave}_3$ , whereas if  $b = 1$  then  $\mathcal{O}$  instead provides freshly generated keys matching the

protocol of  $\text{Leave}_4$ . Thus by forwarding the bit  $b'$  to its own game,  $\mathcal{B}$  wins in the case that  $\mathcal{A}$  was able to identify the game instance correctly.

$$|\varepsilon_4^{\text{leave}} - \varepsilon_3^{\text{leave}}| \leq \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda)$$

By an analogous argument as for distinguishing between  $\text{Leave}_3$  and  $\text{Leave}_4$ , the advantage for  $\text{Leave}_5$  grows by a factor at most that of an adversary against pk-unlinkability of ARKG.

$$|\varepsilon_5^{\text{leave}} - \varepsilon_4^{\text{leave}}| \leq \text{Adv}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(\lambda)$$

In  $\text{Leave}_5$  since both possible revocation tokens  $\tau_0, \tau_1$  that the adversary can receive in the choose phase are identically distributed, it must have probability exactly  $\frac{1}{2}$  of winning.

$$\varepsilon_5^{\text{leave}} = 0$$

Now tracing back the changes in advantage recovers the theorem statement.  $\square$

## 6. Instantiation and Efficiency Analysis

We introduced a generic construction for dynamic group signature construction that mainly relies on ARKG and SFPK. In this section we ensure that our instantiations of those primitives are compatible, and exhibit the necessary trapdoors. We present an adapted version of ARKG from Frymann et al. [26] with trapdoors, we prove in Lemma 1 that it satisfies our new linking soundness property. We also utilise the SFPK scheme from Backes et al. [11]. Both constructions are based on bilinear maps, and we formally present them in Figures 3 and 4.

For the digital signature needed to sign on two dynamic lists:  $\text{pk}$  and  $\text{RL}$ , we rely on the standard signature ECDSA [30] and hash function SHA-3 [31]. To implement this scheme, we hash together the lists as  $h_2 = H_2(\text{pk}, \text{RL})$  and sign on  $h_2$ . This implicitly assumes collision resistance of our chosen hash function. Finally, we use HMAC-SHA256 [32] for the internal MAC used in ARKG.

**Lemma 1** (Linking Soundness of ARKG). *The ARKG protocol shown in Figure 3 with  $\text{pp} = ((\mathbb{G}_1, \mathbb{G}_2, g_1, g_2, q), H, \text{MAC}, \text{KDF}_1, \text{KDF}_2)$  has the linking soundness property.*

*Proof.* Let  $(s, S) \leftarrow \$ \text{ARKG.KeyGen}(\text{pp})$  and consider some  $\text{cred} := (E, \text{aux}, \mu)$ . Suppose  $\text{ChkLink}(S, X, \text{cred}, e) = 1$ , then we have the following.

$$\begin{aligned} \mu &= \text{MAC}(\text{KDF}_2(S^e), (E, \text{aux})) \\ X &= (g_1^{\text{KDF}_1(S^e)}, g_2^{\text{KDF}_1(S^e)}) \cdot S \\ E &= (g_1^e, g_2^e) \end{aligned}$$

Relying on  $S^e = E^s$ , we have:  $\mu = \text{MAC}(\text{KDF}_2(E^s), (E, \text{aux}))$ . Then for  $x \leftarrow$

|                                                                  |                                                         |                                                      |                                                                      |
|------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------|
| <b>Setup</b> ( $\lambda$ )                                       | <b>DerivePK</b> ( $S, aux$ )                            | <b>DeriveSK</b> ( $s, cred$ )                        | <b>ChkLink</b> ( $S, X, cred, e$ )                                   |
| $KDF_1, KDF_2 \leftarrow \$ KDF.Gen(\lambda)$                    | <b>parse</b> $S = (S_1, S_2)$                           | <b>parse</b> $cred = (E, aux, \mu)$                  | <b>parse</b> $S = (S_1, S_2)$                                        |
| $fp \leftarrow (MAC, KDF_1, KDF_2)$                              | $(e, E) \leftarrow KeyGen(pp)$                          | <b>parse</b> $E = (E_1, E_2)$                        | <b>parse</b> $cred = (E, aux, \mu)$                                  |
| $pp \leftarrow (gp, fp)$                                         | $ck \leftarrow KDF_1((S_1^e, S_2^e))$                   | $ck \leftarrow KDF_1((E_1^s, E_2^s))$                | $ck \leftarrow KDF_1((S_1^e, S_2^e))$                                |
| <b>return</b> $pp$                                               | $mk \leftarrow KDF_2((S_1^e, S_2^e))$                   | $mk \leftarrow KDF_2((E_1^s, E_2^s))$                | $mk \leftarrow KDF_2((S_1^e, S_2^e))$                                |
| <b>KeyGen</b> ( $pp$ )                                           | $X \leftarrow (g_1^{ck} \cdot S_1, g_2^{ck} \cdot S_2)$ | $x \leftarrow ck + s$                                | $X' \leftarrow (g_1^{ck} \cdot S_1, g_2^{ck} \cdot S_2)$             |
| $s \leftarrow \$ \mathbb{Z}_q^*$ ; $S \leftarrow (g_1^s, g_2^s)$ | $\mu \leftarrow MAC(mk, (E, aux))$                      | $\mu' \leftarrow MAC(mk, (E, aux))$                  | $\mu' \leftarrow MAC(mk, (E, aux))$                                  |
| <b>return</b> $(s, S)$                                           | $cred \leftarrow (E, aux, \mu)$                         | <b>if</b> $\mu \neq \mu'$ <b>then return</b> $\perp$ | <b>return</b> $\mu \stackrel{?}{=} \mu' \wedge X \stackrel{?}{=} X'$ |
| <b>ChkKeys</b> ( $x, X$ )                                        | <b>return</b> $(X, cred, e)$                            | <b>return</b> $x$                                    | $\wedge ChkKeys(E, e)$                                               |
| $X' = (g_1^x, g_2^x)$                                            |                                                         |                                                      |                                                                      |
| <b>return</b> $X \stackrel{?}{=} X'$                             |                                                         |                                                      |                                                                      |

Figure 3. ARKG algorithms with proof of derivation adapted from Frymann et al. [20] our instantiation.

**DeriveSK**( $s, cred$ ) we have  $x = KDF_1(E^s) + s$  and hence

$$\begin{aligned}
X &= (g_1^{KDF_1(S^e)}, g_2^{KDF_1(S^e)}) \cdot S \\
&= (g_1^{KDF_1(S^e)}, g_2^{KDF_1(S^e)}) \cdot (g_1^s, g_2^s) \\
&= (g_1^{KDF_1(S^e)+s}, g_2^{KDF_1(S^e)+s}) \\
&= (g_1^{KDF_1(E^s)+s}, g_2^{KDF_1(E^s)+s}) \\
&= (g_1^x, g_2^x).
\end{aligned}$$

Therefore  $ChkKeys(x, X) = 1$  as required.  $\square$

We now discuss the adjustments required for the SFPK scheme [11] to be compatible with our DGS-VLR construction. In particular, we modify the SFPK key generation algorithm **TrapGen** to take as input an existing public key primitive  $X$ . It outputs a public key  $pk$  alongside a trapdoor  $\tau$  as before, but it no longer outputs a secret key  $sk$ . Internally, it still samples the first two secrets  $a, b$  as in [11] but no longer samples  $x$ , parsing  $pk = X$  to obtain pre-computed values for  $g_1^x, g_2^x$  instead. This change shifts the responsibility of trapdoor generation away from the signer, deviating slightly from the model of Backes et al. The CRS generation for the vector  $Y$  is separated from the **Setup** algorithm to highlight the requirement for this to be implemented by a trusted party.

### 6.1. Instantiating the Zero-Knowledge Proof

For generality, we have presented our construction that makes use of a zero-knowledge proof to link together a derived key, admitting a user to the group, and the signing key, which has been randomised within an equivalence class. To instantiate this, we consider a zero-knowledge proof that proves the relation  $pk' = ChgPK(pk, r) \wedge pk \in pkl$ . The first equation can be proven using Groth-Sahai proofs [33], which would consist of proving 3 multi-scalar multiplication equations in  $\mathbb{G}_1$  and the latter will be a set membership proof

that the commitment contains a  $pk$  in group member list  $pkl$ , for this we utilise the OR proofs detailed in Ràfols [24].

However, this incurs significant cost from an efficiency point of view, namely the signature size is dependent on the number of group members. This is unsatisfactory so instead observe that Structure-Preserving Signatures on Equivalence Classes (SPS-EQ) [22] can be adopted with keys derived from ARKG. The signing algorithm of SPS-EQ defines an equivalence relation over the message space. It allows a signer to sign on one representative of an equivalence class to create a signature for the whole class. The signature can then be changed without the knowledge of the signing key to a different representative. SPS-EQ signature have been used in existing dynamic group signatures [11], [7], however we note that its function in our scheme does differ to these works. Namely, the group manager creates SPS-EQ keys and outputs the public key as part of  $gmpk$ . Then, during **AddUser**, the group manager creates an SPS-EQ signature over  $pk$  and adds this to the group information info. During signing, the signer forgoes the proof  $\Pi$ , and instead outputs a changed signature that verifies the randomised key  $pk'$ . Intuitively, a verifier is assured that the member must be a valid group member because the unforgeability property of SPS-EQ ensures that, without knowledge of the secret key, the changed signature must have been changed from an existing valid SPS-EQ signature.

### 6.2. Efficiency Analysis

We now present the size of the keys, signature and revocation list in Table 2. We focus on the SPS-EQ certificate approach due to its favourable signature size. Overall, our signature consists of a DS signature (512bits), a public key  $pk'$  ( $1\mathbb{G}_1 + 1\mathbb{G}_2$ ) and a SPS-EQ signature ( $10\mathbb{G}_1 + 4\mathbb{G}_2$ ), thus our total signature size is  $11\mathbb{G}_1 + 5\mathbb{G}_2 + 512\text{bits}$ . The public key consists of 3 elements in  $\mathbb{G}_1$ , whereas our secret only consists of 1 element in  $\mathbb{Z}_q^*$ . Our revocation list stores 1 element from  $\mathbb{G}_2$  and 2 from  $\mathbb{Z}_q^*$  per revoked user.

| CRSGen(pp)                                                      | TrapGen( $X$ )                                                 | ChkRep( $\tau, pk$ )                                                                      |
|-----------------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 1 : <b>parse</b> $pp = (gp, fp)$                                | 1 : <b>parse</b> $X = (X_1, X_2)$                              | 1 : <b>parse</b> $pk = (pk_1, pk_2, pk_3)$                                                |
| 2 : <b>and</b> $gp = (\mathbb{G}_1, \mathbb{G}_2, g_1, g_2, q)$ | 2 : $a, b \leftarrow \mathbb{Z}_q^*$                           | 2 : <b>parse</b> $\tau = (\tau_1, a, b)$                                                  |
| 3 : $y, z \leftarrow \mathbb{Z}_q^*$                            | 3 : $(A, B) \leftarrow (g_1^a, g_1^b)$                         | 3 : $\tau_2 \leftarrow g_2^a; \tau_3 \leftarrow g_2^b$                                    |
| 4 : $h \leftarrow g_1^z$                                        | 4 : $pk \leftarrow (X_1, A, B)$                                | 4 : <b>return</b>                                                                         |
| 5 : $Y \leftarrow (g_1^y, g_2^y, h)$                            | 5 : $\tau \leftarrow (X_2, a, b)$                              | 5 : $\bigwedge_{i=1}^3 \bigwedge_{j=1}^3 e(pk_i, \tau_j) \stackrel{?}{=} e(pk_j, \tau_i)$ |
| 6 : <b>return</b> $Y$                                           | 6 : <b>return</b> $(pk, \tau)$                                 |                                                                                           |
| Sign( $Y, sk = x, pk, m$ )                                      | Verify( $Y, pk, m, \sigma$ )                                   | ChgPK( $pk, r$ )                                                                          |
| 1 : <b>parse</b> $Y = (Y_1, Y_2, h)$                            | 1 : <b>parse</b> $Y = (Y_1, Y_2, h)$                           | 1 : <b>parse</b> $pk = (X_1, A, B)$                                                       |
| 2 : $r, u \leftarrow \mathbb{Z}_q^*$                            | 2 : <b>parse</b> $pk = (X_1, A, B)$                            | 2 : $pk' \leftarrow (X_1^r, A^r, B^r)$                                                    |
| 3 : $v \leftarrow H(m g_1^r g_2^r pk)$                          | 3 : <b>parse</b> $\sigma = (\sigma_1, \sigma_2, \sigma_3, u)$  | 3 : <b>return</b> $pk'$                                                                   |
| 4 : $M \leftarrow g_1^v \cdot h^u$                              | 4 : $v \leftarrow H(m \sigma_2 \sigma_3 pk)$                   |                                                                                           |
| 5 : $R \leftarrow Y_1^x \cdot (H(M))^r$                         | 5 : $M \leftarrow g_1^v \cdot h^u$                             | <b>ChgSK</b> ( $sk = x, r$ )                                                              |
| 6 : $\sigma \leftarrow (R, g_1^r, g_2^r, u)$                    | 6 : <b>return</b> $e(\sigma_2, g_2) = e(g_1, \sigma_3) \wedge$ | 1 : $sk' \leftarrow x \cdot r$                                                            |
| 7 : <b>return</b> $\sigma$                                      | 7 : $e(\sigma_1, g_2) = e(X_1, Y_2) \cdot e(H(M), \sigma_3)$   | 2 : <b>return</b> $sk'$                                                                   |

Figure 4. SFPK algorithms adapted for pre-generated key pairs  $(sk, pk) = (x, X)$ , where  $H$  is a collision-resistant hash function with image  $\mathbb{G}_1$ .

Table 2. COMPARISON AGAINST EXISTING DYNAMIC GROUP SIGNATURES. WE USE  $pk$  TO DENOTE THE USE OF THE USER PUBLIC KEY,  $usk$  FOR THEIR SECRET KEY,  $\Sigma$  FOR THE SIZE OF THE SIGNATURE AND  $RL$  FOR THE REVOCATION LIST, WHERE  $r$  IS THE NUMBER OF ENTRIES IN THE LIST. REVOKE SHOWS ASYMPTOTIC RUNTIME FOR REVOKING  $r$  OUT OF  $n$  GROUP MEMBERS. ZK PROOF SIZE<sup>1</sup> COMPUTED USING [24].

| Scheme            | pk              | usk               | $\Sigma$                                                  | RL                                   | Revoke           |
|-------------------|-----------------|-------------------|-----------------------------------------------------------|--------------------------------------|------------------|
| Backes et al. [7] | $2\mathbb{G}_1$ | $3\mathbb{G}_1$   | $28\mathbb{G}_1 + 15\mathbb{G}_2 + 512\text{bits}$        | N/A                                  | $\mathcal{O}(n)$ |
| DGS-VLR w/ NIZK   | $3\mathbb{G}_1$ | $1\mathbb{Z}_q^*$ | $(4n + 21)\mathbb{G}_1 + 19\mathbb{G}_2 + 512\text{bits}$ | $r(1\mathbb{G}_2 + 2\mathbb{Z}_q^*)$ | $\mathcal{O}(1)$ |
| DGS-VLR w/ SPS-EQ | $3\mathbb{G}_1$ | $1\mathbb{Z}_q^*$ | $11\mathbb{G}_1 + 5\mathbb{G}_2 + 512\text{bits}$         | $r(1\mathbb{G}_2 + 2\mathbb{Z}_q^*)$ | $\mathcal{O}(1)$ |

Compared to Backes et al. [7], we achieve the smaller signature size despite use of the same underlying use of the SFPK scheme. This is due to the fact that our VLR approach means we can reduce the complexity of the relation for the proof that comprises part of the signature. However, our main advantage comes from the fact we have dropped the notion of epochs to manage revocation and joining within the scheme. In particular, to process a group update, our scheme publishes the group information info which has size  $512(n+1)\text{bits} + 3n\mathbb{G}_1 + (3n+r)\mathbb{G}_2 + 2r\mathbb{Z}_q^*$ , where  $n$  is the number of group members and  $r$  is the number of revoked users. As mentioned previously, this update is the same for all users and can be published globally. This compares to Backes et al. where each group member needs to regenerate its signing key, where user-specific computation consists of generating ciphertexts which dominate this phase. The size of the update is  $10\mathbb{G}_1 + 14\mathbb{G}_2 + 512\text{bits}^2 + 2\mathbb{G}^3$  per group member. Additionally, to perform revocation (i.e., calling Revoke) is more efficient for VLR schemes  $\mathcal{O}(1)$  compared to schemes on secret credentials  $\mathcal{O}(n)$  for  $n$  group members.

2. The original paper does not specify which digital signature the authors instantiate their scheme with, so we use ECDSA for fair comparison.

3. This term is derived from the author's use of the ElGamal encryption scheme, for which the underlying group can be independently chosen of  $\mathbb{G}_1$  and  $\mathbb{G}_2$ , so we do not specify further.

## 7. Conclusion

In this paper we have proposed a Dynamic Group Signature scheme with a Verifier-Local Revocation – the first dynamic group signature scheme with membership privacy to benefit from the efficiency improvements for using a VLR approach. We were able to achieve this by adapting Asynchronous Remote Key Generation to also support a linking trapdoor, for which we defined a new security property *Linking Soundness*. When combined with a trapdoor from Signatures with Flexible Public Keys to form an extended two-stage tracing process, this allowed the latter trapdoor to serve as the revocation token in our generic construction without compromising membership privacy. We have instantiated our generic scheme with pairing-based building blocks to give a DGS-VLR that is competitive with the state-of-the-art from an efficiency point of view, but with improved functionality. By adopting a VLR approach, our scheme avoids reissuing keys during group updates and thus enables more efficient joining and leaving. Finally, when instantiated with SPS-EQ our signatures each consist of  $11\mathbb{G}_1 + 5\mathbb{G}_2 + 512\text{bits}$ , which is smaller than all other dynamic schemes to date (cf. Table 2).

*Future Work.* Whilst we have demonstrated favourable asymptotic performance, a concrete implementation would provide better metrics for real world use. Furthermore, post-quantum security guarantees would be an additional highly desirable property. This could be achieved through a lattice-

based implementation; in fact, lattice-based constructions have already been proposed for ARKG [27], [28], [29]. However, these implementations do not support the necessary trapdoor for *Linking Soundness*. Additionally, SPS-EQ from lattices is also an open problem, both of which would be required to extend our results to achieve PQ security.

## Acknowledgments

C. C. Drăgan is partially supported by TrustVote – EPSRC grant EP/Y020529/1, AP4L - EPSRC grant EP/W032473/1, CONNECT - Horizon Europe Guarantee 10043730 and EU Horizon grants 101069688, REWIRE - Horizon Europe Guarantee 10043743 and EU Horizon grants 101070627.

## References

- [1] D. Chaum and E. van Heyst, “Group Signatures,” in *Proceedings of EUROCRYPT 1991*, vol. 547 of *LNCS*, pp. 257–265, Springer, 1991.
- [2] A. Lysyanskaya and Z. Ramzan, “Group blind digital signatures: A scalable solution to electronic cash,” in *Financial Cryptography, Second International Conference, FC’98, Anguilla, British West Indies, February 23–25, 1998, Proceedings*, vol. 1465 of *LNCS*, pp. 184–197, Springer, 1998.
- [3] S. Canard and J. Traoré, “On fair e-cash systems based on group signature schemes,” in *Information Security and Privacy, 8th Australasian Conference, ACISP 2003, Wollongong, Australia, July 9–11, 2003, Proceedings*, vol. 2727 of *LNCS*, pp. 237–248, Springer, 2003.
- [4] M. Bellare, D. Micciancio, and B. Warinschi, “Foundations of Group Signatures: Formal Definitions, Simplified Requirements, and a Construction Based on General Assumptions,” in *Proceedings of EUROCRYPT 2003*, vol. 2656 of *LNCS*, pp. 614–629, Springer, 2003.
- [5] M. Bellare, H. Shi, and C. Zhang, “Foundations of Group Signatures: The Case of Dynamic Groups,” in *Proceedings of CT-RSA 2005*, vol. 3376 of *LNCS*, pp. 136–153, Springer, 2005.
- [6] J. Bootle, A. Cerulli, P. Chaidos, E. Ghadaoui, and J. Groth, “Foundations of Fully Dynamic Group Signatures,” in *Proceedings of ACNS 2016*, vol. 9696 of *LNCS*, pp. 117–136, Springer, 2016.
- [7] M. Backes, L. Hanzlik, and J. Schneider-Bensch, “Membership Privacy for Fully Dynamic Group Signatures,” in *Proceedings of the CCS 2019*, pp. 2181–2198, ACM, 2019.
- [8] “DocuSign group signing.” [https://support.docusign.com/s/document-item?language=en\\_US&rsc\\_301&bundleId=gav1643676262430&topicId=zgn1578456447934.html](https://support.docusign.com/s/document-item?language=en_US&rsc_301&bundleId=gav1643676262430&topicId=zgn1578456447934.html). Accessed on 17th April 2025.
- [9] B. Brecht, D. Theriault, A. Weimerskirch, W. Whyte, V. Kumar, T. Hehn, and R. Goudy, “A security credential management system for v2x communications,” *IEEE Transactions on Intelligent Transportation Systems*, vol. PP, 02 2018.
- [10] J. Petit, F. Schaub, M. Feiri, and F. Kargl, “Pseudonym schemes in vehicular networks: A survey,” *Commun. Surveys Tuts.*, vol. 17, p. 228–255, Jan. 2015.
- [11] M. Backes, L. Hanzlik, K. Klucznik, and J. Schneider, “Signatures with Flexible Public Key: Introducing Equivalence Classes for Public Keys,” in *Proceedings of Part II of ASIACRYPT 2018*, vol. 11273 of *LNCS*, pp. 405–434, Springer, 2018.
- [12] D. Boneh and H. Shacham, “Group signatures with verifier-local revocation,” in *Proceedings of CCS 2004*, pp. 168–177, ACM, 2004.
- [13] P. Bichsel, J. Camenisch, G. Neven, N. P. Smart, and B. Warinschi, “Get Shorty via Group Signatures without Encryption,” in *Proceedings of SCN 2010*, vol. 6280 of *LNCS*, pp. 381–398, Springer, 2010.
- [14] A. Ishida, Y. Sakai, K. Emura, G. Hanaoka, and K. Tanaka, “Fully Anonymous Group Signature with Verifier-Local Revocation,” in *Proceedings of SCN 2018*, vol. 11035 of *LNCS*, pp. 23–42, Springer, 2018.
- [15] B. Libert, T. Peters, and M. Yung, “Scalable group signatures with revocation,” in *Proceedings of EUROCRYPT 2012*, vol. 7237 of *LNCS*, pp. 609–627, Springer, 2012.
- [16] T. Nakanishi, H. Fujii, Y. Hira, and N. Funabiki, “Revocable Group Signature Schemes with Constant Costs for Signing and Verifying,” in *Proceedings of PKC 2009*, vol. 5443 of *LNCS*, pp. 463–480, Springer, 2009.
- [17] V. Kumar, H. Li, N. Luther, P. Asokan, J.-M. J. Park, K. Bian, M. B. H. Weiss, and T. Znati, “Direct anonymous attestation with efficient verifier-local revocation for subscription system,” in *Proceedings of the 2018 on Asia Conference on Computer and Communications Security, ASIACCS ’18*, p. 567–574, Association for Computing Machinery, 2018.
- [18] Y. NAKAZAWA and T. NAKANISHI, “A strongly unlinkable group signature scheme with matching-based verifier-local revocation for privacy-enhancing crowdsensing,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E106.A, no. 12, pp. 1531–1543, 2023.
- [19] K. Banti, F. Katsimpoura, M. Louta, and G. T. Karetsos, “Data quality in mobile crowd sensing systems: Challenges and perspectives,” in *2018 9th International Conference on Information, Intelligence, Systems and Applications (IISA)*, pp. 1–8, 2018.
- [20] N. Frymann, D. Gardham, F. Kiefer, E. Lundberg, M. Manulis, and D. Nilsson, “Asynchronous Remote Key Generation: An Analysis of Yubico’s Proposal for W3C WebAuthn,” in *Proceedings of CCS 2020*, pp. 939–954, ACM, 2020.
- [21] J. Katz, *Digital Signatures*. Springer, 2010.
- [22] G. Fuchsbaauer and R. Gay, “Weakly secure equivalence-class signatures from standard assumptions,” in *Public-Key Cryptography – PKC 2018*, pp. 153–183, Springer International Publishing, 2018.
- [23] B. Libert, T. Peters, and M. Yung, “Group Signatures with Almost-for-Free Revocation,” in *Proceedings of CRYPTO 2012*, vol. 7417 of *LNCS*, pp. 571–589, Springer, 2012.
- [24] C. Ràfols, “Stretching groth-sahai: Nizk proofs of partial satisfiability,” in *Theory of Cryptography*, pp. 247–276, Springer Berlin Heidelberg, 2015.
- [25] N. Frymann, D. Gardham, and M. Manulis, “Unlinkable Delegation of WebAuthn Credentials,” in *Proceedings of PART III of ESORICS 2022*, vol. 13556 of *LNCS*, pp. 125–144, Springer, 2022.
- [26] N. Frymann, D. Gardham, M. Manulis, and H. Nartz, “Generalised Asynchronous Remote Key Generation for Pairing-Based Cryptosystems,” in *Proceedings of Part I of ACNS 2023*, vol. 13905 of *LNCS*, pp. 394–421, Springer, 2023.
- [27] N. Frymann, D. Gardham, and M. Manulis, “Asynchronous Remote Key Generation for Post-Quantum Cryptosystems from Lattices,” in *Proceedings of IEEE EuroS&P 2023*, pp. 928–941, IEEE, 2023.
- [28] D. Stebila and S. Wilson, “Quantum-safe account recovery for webauthn,” in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2024, Singapore, July 1–5, 2024*, ACM, 2024.
- [29] J. Brendel, S. Clermont, and M. Fischlin, “Post-quantum asynchronous remote key generation for fido2,” in *ASIACRYPT 2024*, Springer-Verlag, 2024.
- [30] D. Johnson, A. Menezes, and S. A. Vanstone, “The Elliptic Curve Digital Signature Algorithm (ECDSA),” *Int. J. Inf. Sec.*, vol. 1, no. 1, pp. 36–63, 2001.
- [31] National Institute of Standards and Technology (NIST), “SHA-3 standard: Permutation-based hash and extendable-output functions,” 2015.

- [32] H. Krawczyk, M. Bellare, and R. Canetti, “Rfc2104: Hmac: Keyed-hashing for message authentication,” 1997.
- [33] J. Groth and A. Sahai, “Efficient non-interactive proof systems for bilinear groups,” in *Advances in Cryptology – EUROCRYPT 2008*, pp. 415–432, Springer Berlin Heidelberg, 2008.

## **Appendix A.**

### **Building Blocks**

Here we recall additional building blocks and their security properties.

The security experiments for SFPK are given in Figure 5. Note that the inclusion of  $\sigma^*$  in line 5 of the euf-cma experiment transforms it into the suf-cma (strong euf-cma) experiment.

The security experiments for ARKG are given in Figure 6. The presented definitions have been modified to capture the trapdoor  $\tau$  as an additional output from algorithm DerivePK.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Exp}_{\mathcal{A}, \text{SFPK}}^{\text{ch}}(1^\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | $\text{Exp}_{\mathcal{A}, \text{SFPK}}^{\text{euf-cma}}(1^\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                          | $\mathcal{O}_{\text{unfSign1}}(m)$                                                                                                                                                                                   |
| 1 : $\text{pp} \leftarrow \$ \text{Setup}(\lambda)$<br>2 : $(\text{sk}_0, \text{pk}_0) \leftarrow \$ \text{KeyGen}()$<br>3 : $(\text{sk}_1, \text{pk}_1) \leftarrow \$ \text{KeyGen}()$<br>4 : $b \leftarrow \$ \{0, 1\}; r \leftarrow \$ \mathbb{Z}_q^*$<br>5 : $\text{pk}' \leftarrow \text{ChgPK}(\text{pk}_b, r)$<br>6 : $\text{sk}' \leftarrow \text{ChgSK}(\text{sk}_b, r)$<br>7 : $\text{inp} \leftarrow ((\text{sk}_0, \text{pk}_0), (\text{sk}_1, \text{pk}_1), \text{pk}')$<br>8 : $b' \leftarrow \$ \mathcal{A}^{\mathcal{O}_{\text{chSign1}}, \mathcal{O}_{\text{chSign2}}}(\text{pp}, \text{inp})$<br>9 : <b>return</b> $b = b'$ | 1 : $\text{pp} \leftarrow \$ \text{Setup}(\lambda)$<br>2 : $Q_{\text{sig}} \leftarrow \emptyset$<br>3 : $(\text{sk}, \text{pk}, \tau) \leftarrow \$ \text{TrapGen}()$<br>4 : $(\text{pk}^*, m^*, \sigma^*) \leftarrow \$ \mathcal{A}^{\mathcal{O}_{\text{unfSign1}}, \mathcal{O}_{\text{unfSign2}}}(\text{pk}, \tau)$<br>5 : <b>return</b> $(m^*, [\sigma^*]_{-1}) \notin Q_{\text{sig}}$<br>6 : $\wedge \text{ChkRep}(\tau, \text{pk}^*) = 1$<br>7 : $\wedge \text{Verify}(\text{pk}^*, m^*, \sigma^*) = 1$ | 1 : $\sigma \leftarrow \$ \text{Sign}(\text{sk}, m)$<br>2 : $Q_{\text{sig}} \leftarrow Q_{\text{sig}} \cup \{(m, \sigma)\}$<br>3 : <b>return</b> $\sigma$                                                            |
| $\mathcal{O}_{\text{chSign1}}(m)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | $\mathcal{O}_{\text{chSign2}}(m, r')$                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | $\mathcal{O}_{\text{unfSign2}}(m, r)$                                                                                                                                                                                |
| $\sigma \leftarrow \$ \text{Sign}(\text{sk}', m)$<br><b>return</b> $\sigma$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | $\text{sk}'' \leftarrow \text{ChgSK}(\text{sk}', r')$<br>$\sigma \leftarrow \$ \text{Sign}(\text{sk}'', m)$<br><b>return</b> $\sigma$                                                                                                                                                                                                                                                                                                                                                                        | 1 : $\text{sk}' \leftarrow \text{ChgSK}(\text{sk}, r)$<br>2 : $\sigma \leftarrow \$ \text{Sign}(\text{sk}', m)$<br>3 : $Q_{\text{sig}} \leftarrow Q_{\text{sig}} \cup \{(m, \sigma)\}$<br>4 : <b>return</b> $\sigma$ |

Figure 5. Experiments for SFPK class-hiding and (strong) unforgeability.

|                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{pku}}(1^\lambda)$                                                                                                                                                                                                                          | $\text{Exp}_{\mathcal{A}, \text{ARKG}}^{\text{skS}}(1^\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | $\mathcal{O}_{\text{dpk}}(\text{pk}, \text{aux})$                                                                                                                                                                                                   |
| 1 : $\text{pp} \leftarrow \$ \text{Setup}(\lambda)$<br>2 : $(\text{sk}, \text{pk}) \leftarrow \$ \text{KeyGen}()$<br>3 : $b \leftarrow \$ \{0, 1\}$<br>4 : $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{pku}}}(\text{pk})$<br>5 : <b>return</b> $b' = b$                               | 1 : $\text{pp} \leftarrow \$ \text{Setup}(\lambda)$<br>2 : $Q_{\text{pk}'}, Q_{\text{sk}'} \leftarrow \emptyset$<br>3 : $(\text{sk}, \text{pk}) \leftarrow \$ \text{KeyGen}(\text{pp})$<br>4 : $(\text{sk}^*, \text{pk}^*, \text{cred}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{dpk}}, \mathcal{O}_{\text{dsk}}}(\text{pk})$<br>5 : $\text{sk}' \leftarrow \text{DeriveSK}(\text{sk}, \text{cred}^*)$<br>6 : <b>return</b> $\text{ChkKeys}(\text{pp}, \text{sk}^*, \text{pk}^*)$<br>7 : $\wedge \text{ChkKeys}(\text{pp}, \text{sk}', \text{pk}^*)$<br>8 : $\wedge \text{cred}^* \notin Q_{\text{sk}'}$ | 1 : $(\text{pk}', \text{cred}, \tau) \leftarrow \text{DerivePK}(\text{pk}, \text{aux})$<br>2 : $Q_{\text{pk}'} \leftarrow Q_{\text{pk}'} \cup \{(\text{pk}', \text{cred})\}$<br>3 : <b>return</b> $(Q_{\text{pk}'}, \text{pk}', \text{cred}, \tau)$ |
| $\mathcal{O}_{\text{pku}}()$                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | $\mathcal{O}_{\text{dsk}}(\text{sk}, \text{cred})$                                                                                                                                                                                                  |
| 1 : $(\text{pk}_0, \text{cred}, \tau) \leftarrow \$ \text{DerivePK}(\text{pk})$<br>2 : $\text{sk}_0 \leftarrow \text{DeriveSK}(\text{sk}, \text{cred})$<br>3 : $(\text{sk}_1, \text{pk}_1) \leftarrow \$ \text{KeyGen}()$<br>4 : <b>return</b> $(\text{sk}_b, \text{pk}_b, \text{cred})$ |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 1 : $\text{sk}' \leftarrow \text{DeriveSK}(\text{sk}, \text{cred})$<br>2 : $Q_{\text{sk}'} \leftarrow Q_{\text{sk}'} \cup \{\text{cred}\}$<br>3 : <b>return</b> $(Q_{\text{sk}'}, \text{sk}')$                                                      |

Figure 6. Experiments for ARKG pk-unlinkability and sk-secrecy.